

Міністерство освіти і науки України

Черкаський національний університет імені Богдана Хмельницького Факультет
обчислювальної техніки, інтелектуальних та управляючих систем

Кафедра програмного забезпечення автоматизованих систем

КУРСОВА РОБОТА З ОБ'ЄКТНО-ОРІЄНТОВАНОГО ПРОГРАМУВАННЯ

на тему «Симулятор системи управління смарт-будинком»

Студента(ки) 2 курсу, групи КС-231
спеціальності 121 “Інженерія програмного
забезпечення”

Тукала І. В.

(прізвище та ініціали)

Керівник: к. т. н., доцент Ярмілко А. В.

(посада, вчене звання, науковий ступінь, прізвище та ініціали)

Оцінка за шкалою:

(національною, кількість балів, ECTS)

Члени комісії:

(підпис)(прізвище та ініціали)

(підпис)(прізвище та ініціали)

(підпис)(прізвище та ініціали)

Черкаси – 2025 рік

Зміст

Вступ	3
Розділ 1. Огляд алгоритмів, методів та аналогічних систем управління смарт-будинками	5
1.1 Комерційні закриті екосистеми управління смарт-будинками.....	5
1.2 Відкриті програмні платформи управління смарт-будинками.....	6
1.3 Огляд та вибір алгоритмів і методів реалізації симулятора системи управління смарт-будинком	7
1.3.1 Аналіз підходів до побудови графічного інтерфейсу (GUI)	7
1.3.2 Аналіз підходів до моделювання симуляції	8
1.3.3 Алгоритм управління логікою пристроїв (реалізація ООП)	8
1.3.4 Алгоритм зв'язку логіки та візуалізації (UI)	9
1.4 Постановка задачі	10
1.5 Висновки до першого розділу	10
Розділ 2. Проєктування архітектури симулятора системи управління смарт- будинком	12
2.1 Вимоги до симулятора системи управління смарт-будинком	12
2.2 Проєктування структури симулятора системи управління смарт- будинком	13
2.2.1 Компоненти, які включає в себе симулятор системи управління смарт-будинком	13
2.2.2 Потік даних всередині симулятора системи управління смарт- будинком	14
2.3. Проєктування об'єктно-орієнтованої моделі симулятора системи управління смарт-будинком	15
2.3.1 Базовий абстрактний клас	15
2.3.2 Спеціалізовані групи та класи.....	16
2.4 Опис основних алгоритмів функціонування системи	18
2.5 Висновок до другого розділу	23

Розділ 3. Програмна реалізація та тестування симулятора системи управління смарт-будинком	25
3.1 Обґрунтування вибору засобів розробки.....	25
3.2 Розробка компонентів симулятора.....	26
3.3 Застосування наслідування, інкапсуляції та поліморфізму в проектуванні симулятора системи управління смарт-будинком.....	27
3.4 Тестування симулятора системи управління смарт-будинком.....	28
3.5 Висновки до третього розділу	40
Висновки	42
Список використаних джерел.....	44
Додатки.....	48
Додаток А. Словник ключових термінів проєкту	48
Додаток Б. Список усіх областей приладів та об'єктів	50
Додаток В. Повний код проєкту	53
Додаток Г. Діаграми класів.....	54

Вступ

Сучасне життя є досить динамічним. Людина щодня стикається з різноманітними подразниками, що вимагають оперативної та ефективної реакції. У такому ритмі надзвичайно важливим стає життєвий простір – квартира чи будинок, – де можна почуватися комфортно та безпечно.

Цей комфорт забезпечується певними базовими характеристиками середовища, які безпосередньо пов'язані з фундаментальними потребами людини, що відображені в діаграмі Маслоу, такими як фізіологічний комфорт (оптимальна температура, вологість, освітлення) та потреба у безпеці. Створення та підтримка такого побутового середовища є однією з умов для здорового, щасливого та продуктивного життя.

Сучасні технології дають можливість автоматизувати контроль над цими характеристиками за допомогою систем, що отримали загальну назву «розумний дім». Такі системи інтегрують різноманітні пристрої житлового приміщення, щоб забезпечити не лише комфорт та безпеку мешканців, але й енергоефективність самого будинку. Однак проектування та тестування логіки взаємодії десятків таких пристроїв є складним завданням. Тому в процесі впровадження систем розумного будинку виникає потреба дослідження сценаріїв його функціонування та можливих конфігурації застосованого обладнання. Саме тому виникає потреба у розробці програмних симуляторів, здатних імітувати роботу таких систем. Симулятор, що є предметом даної курсової роботи, якраз і призначений для моделювання та тестування логіки смарт-будинку. Отже, його розробка є актуальною задачею.

Метою даної курсової роботи є проектування та реалізація симулятора системи управління смарт-будинком з використанням принципів об'єктно-орієнтованого програмування, набуття практичних навичок з розробки програмних систем.

Для досягнення поставленої мети необхідно вирішити наступні **завдання**:

1. Провести аналіз предметної області, дослідити наявні літературні джерела та оглянути наявні програмні продукти-симулятори розумного будинку для виявлення основних підходів та алгоритмів.

2. Сформулювати постановку задачі: деталізувати вимоги до функціоналу симулятора, описати його майбутні функції та бізнес-логіку.

3. Виконати проєктування архітектури програмного продукту. Спроєктувати об'єктно-орієнтовану модель системи: розробити ієрархію класів для смарт-пристроїв, обов'язково застосувавши принципи наслідування, інкапсуляції та поліморфізму.

4. Реалізувати розроблений симулятор мовою високого рівня, створити графічний інтерфейс користувача.

5. Описати ключові програмні рішення та фрагменти коду, побудувати діаграму класів реалізованої системи та провести тестування функціоналу шляхом імітації типових сценаріїв.

В представленій роботі використано такі базові поняття:

1. Система «розумний дім» – це програмно-апаратний комплекс, що об'єднує інженерні системи будинку (освітлення, клімат, безпеку) в єдину мережу. Фундаментально, він складається з датчиків (сенсорів), які збирають дані про середовище (напр., температура, рух), та виконавчих пристроїв (актуаторів), які виконують команди (напр., увімкнути обігрівач).

2. Програмний симулятор є комп'ютерною моделлю, яка імітує поведінку та логіку такої реальної системи. Він дозволяє розробнику віртуально розміщувати пристрої, налаштовувати їхню взаємодію та тестувати різні сценарії («що-якщо») без необхідності розгортання дорогого фізичного обладнання.

Розділ 1. Огляд алгоритмів, методів та аналогічних систем управління смарт-будинками

Сучасний ринок систем розумного будинку можна умовно поділити на дві категорії: комерційні закриті екосистеми (орієнтовані на кінцевого споживача) та відкриті платформи (орієнтовані на ентузіастів та гнучке налаштування) [1, 2].

1.1 Комерційні закриті екосистеми управління смарт-будинками

Комерційні закриті екосистеми управління смарт-будинками є платформами, які вирізняються простотою налаштування та орієнтацією на голосове управління, але мають обмежену логіку автоматизації:

1. Amazon Alexa – це платформа, яка базується на використанні голосового асистента та смарт-колонок Echo, є однією з найбільш поширених на ринку [3]. Її ключовою перевагою є найширша сумісність з тисячами пристроїв від сторонніх виробників. Ця відкритість дозволяє користувачам легко налаштовувати базові сценарії автоматизації через візуальний інтерфейс «Рутин» (Routines). Але, логіка автоматизації, доступна користувачеві, залишається досить примітивною, що ускладнює створення складних сценаріїв з багатьма умовами. Також певним недоліком системи є її критична залежність від Інтернету, через що більшість автоматизацій припиняє працювати.

2. Google Home – ця екосистема, інтегрована в Google Assistant, виступає прямим конкурентом для Amazon Alexa [4]. Унікальною рисою цієї платформи є глибока інтеграція з іншими сервісами Google, такими як Календар чи Карти. Алгоритми цієї системи є одними із найкращих для розпізнавання природної мови, через що голосове управління нею є більш гнучке. Google Home має потужний редактор скриптів, що надає можливості автоматизації для просунутих користувачів. Проте, за загальною кількістю сумісних пристроїв Google Home все ще певним чином поступається Amazon Alexa. Як і конкурент, має дещо критичну залежність від Інтернету.

3. Apple HomeKit – це екосистеми, в якій компанія Apple зробила ключовий акцент на безпеці та приватності користувача [5]. На відміну від конкурентів, які для обробки логіки алгоритмів здебільшого покладаються на хмарну інфраструктуру, значна частина автоматизацій HomeKit обробляється локально на пристрої-хабі (такому як Apple TV або HomePod). Це мінімізує відправку даних на зовнішні сервери, підвищуючи надійність та швидкість реакції системи. Підвищення безпеки даної екосистеми призводить до її закритості, тобто змушує виробників проходити сувору сертифікацію (MFi), тому вибір сумісних пристроїв значно вужчий, ніж у конкурентів, а їхня вартість, як правило, вища.

1.2 Відкриті програмні платформи управління смарт-будинками

Відкриті програмні платформи управління смарт-будинками надають майже необмежену гнучкість в функціоналі, однак вимагають достатньо високої технічної підготовленості користувачів та готовності витратити час на налаштування.

1. Home Assistant (HASS) – це програмне забезпечення, характерною особливістю якого є його автономність [6]. Найчастіше воно встановлюється на локальний міні-комп'ютер (наприклад, Raspberry Pi). Це надає користувачеві повний контроль над своїми даними та забезпечує роботу системи навіть без Інтернету. Потужне програмне забезпечення цієї платформи гарантує надійну автоматизацію її процесів, що ними вона керує, дозволяючи через конфігураційні файли (YAML) або візуальний редактор створювати сценарії будь-якої мислимої складності. HASS має високий поріг входження.

2. openHAB – це платформа з відкритим кодом, яка написана на Java [7]. Її архітектура, заснована на «біндингах» (додатках для підтримки пристроїв), є дуже модульною. Але цей підхід робить поріг входження ще вищим, ніж у Home Assistant. Процес конфігурації є складнішим та менш інтуїтивним для новачка через текстові файли.

1.3 Огляд та вибір алгоритмів і методів реалізації симулятора системи управління смарт-будинком

1.3.1 Аналіз підходів до побудови графічного інтерфейсу (GUI)

Для розробки GUI симулятора системи управління смарт-будинку існують два фундаментальні підходи:

1. Традиційні (імперативні) бібліотеки GUI [8] (наприклад, WinForms [9], Java Swing [10], GDI+ [11]). Їхня перевага полягає у простоті та низькому порозі входження, що дозволяє швидко створювати стандартні інтерфейси з кнопками та текстовими полями. Але, вони погано пристосовані для складних, кастомних графічних завдань. Для реалізації 2D-плану будинку з накладанням інтерактивних напівпрозорих областей такий підхід вимагав би складного ручного промальовування всіх елементів через графічні примітиви (напр., `Graphics.DrawRectangle()`). Це є громіздким, неефективним в плані продуктивності методом, який також значно ускладнює подальшу обробку інтерактивності (натискань на ці намальовані області).

2. Декларативні графічні фреймворки [12] (наприклад, WPF [13], JavaFX [14], QML [15]). Вони надають потужну векторну графічну систему та апаратне прискорення. Такі технології дозволяють легко реалізувати 2D-план, розміщуючи елементи (об'єкти-прямокутники) поверх зображення на спеціальній «поверхні» (Canvas [16]). Ключовими перевагами є чітке розділення логіки та представлення (завдяки декларативним мовам розмітки, як XAML [17] або FXML) та, особливо, вбудований механізм прив'язки даних (Binding [18]). Цей механізм дозволяє «прив'язати» візуальну властивість (напр., колір рамки) до логічної властивості (CurrentState) у класі пристрою, що забезпечує автоматичне оновлення UI. Єдиним недоліком цього підходу є дещо вищий поріг входження через необхідність розуміння декларативної розмітки та концепції прив'язки даних.

1.3.2 Аналіз підходів до моделювання симуляції

Наше завдання – це симуляція, тобто модель системи, що змінюється в часі. Існує два фундаментальні підходи для реалізації «рушія» такої симуляції:

1. Симуляція, керована подіями [19]. Даний підхід моделює час нелінійно. Система веде «календар» майбутніх подій. Рушій перестрибує від однієї події до іншої, ігноруючи час між ними. Метод є дуже ефективним для логістичних або мережевих симуляцій. Але він не підходить для даного завдання, оскільки потрібна постійна візуалізація та плавний плин часу. Також нам потрібно моделювати безперервні процеси (зміна температури, вологості та ін).

2. Симуляція з кроком по часу [20], що використовується у більшості ігор та візуальних симуляторів. Час просувається фіксованими «кроками» або «тіками». На кожному кроці система оновлює стан усіх об'єктів.

1.3.3 Алгоритм управління логікою пристроїв (реалізація ООП)

Ядром будь-якої системи є алгоритми автоматизації. Найпоширенішою є модель «Подія-Умова-Дія» (Event-Condition-Action, або ECA) [21]:

1. подія (Event): Тригер, що запускає перевірку (наприклад, датчик зафіксував рух, зміну температури або вологості тощо);
2. умова (Condition): додаткова перевірка стану системи (наприклад, чи час між 16:00 та 8:00?);
3. дія (Action): виконання команди, якщо умова істинна (наприклад, увімкнути світло).

Більш складним підходом є «Машина станів» [22], де вся система або її частини, може перебувати у певних глобальних станах («Вдома», «Ніч», «Відпустка», «Тривога»). Замість налаштування десятків ECA-правил, логіка збігається з переходами між цими станами. Наприклад, перехід у стан «Тривога» автоматично активує сирени і викликає служби охорони.

Для програмної реалізації цих алгоритмів (зокрема ECA), дотримуючись вимог ООП, можна також виділити два підходи:

4. Централізований алгоритм [23] – полягає в створення одного великого класу (напр., `SimulationManager`), який у головному циклі перевіряє стан всіх пристроїв і приймає рішення за них. Даний підхід є прямим порушенням принципів ООП (інкапсуляції та розподілу відповідальності). Код стає нечитабельним, негнучким, а додавання нового пристрою вимагатиме зміни цього алгоритму.

5. Децентралізований алгоритм (із застосуванням поліморфізму) [24]. Даний підхід ґрунтується створенні абстрактного базового класу (напр., `SmartDeviceBase`), який визначає принцип – абстрактний метод `UpdateState()`. Кожен конкретний пристрій (напр., `HeaterDevice`, `CameraDevice`) наслідує цей клас та реалізує власну логіку всередині методу `UpdateState()`. Це чистий та гнучкий підхід. Головний рушій симуляції в циклі не знає нічого про логіку обігрівачів чи камер. Він просто виконує поліморфний виклик `device.UpdateState()` для кожного об'єкта у списку. Обігрівач сам перевірить температуру, а камера сама перевірить таймер руху.

1.3.4 Алгоритм зв'язку логіки та візуалізації (UI)

Необхідно обрати метод, що пов'яже зміну логічного стану пристрою зі зміною його візуального стану:

1. Алгоритм прямого ручного управління [25]. Логічний клас пристрою при зміні свого стану мав би сам знаходити у UI-дереві потрібний графічний елемент (напр., за іменем «`O1_Rect`») і вручну викликати метод для зміни його властивості. Цей підхід створює жорстку зв'язаність між логікою та інтерфейсом, що ускладнює підтримку.

2. Алгоритм прив'язки даних (`Binding`). Це декларативний підхід, доступний у сучасних фреймворках (див. 1.3.1). У файлі розмітки один раз вказується: «Колір рамки цього елемента прив'язаний до властивості `CurrentState`». Логічний клас пристрою нічого «не знає» про UI; він просто змінює свою власну властивість. Платформа автоматично «помічає» цю зміну

(через відповідний інтерфейс, напр. `INotifyPropertyChanged`) і сама оновлює колір рамки. Це забезпечує чисте розділення логіки та представлення.

1.4 Постановка задачі

Ідея програмного проєкту полягає у створенні легковагового симулятора, який візуально моделює та тестує логіку взаємодії пристроїв розумного будинку. Метою є не управління фізичним обладнанням, а швидке проєктування та візуальне налагодження складних сценаріїв автоматизації. На основі проведеного аналізу методів та засобів розробки симуляторів, формулюється наступне завдання на розробку:

1. В якості «рушія» симуляції реалізувати алгоритм симуляції з кроком по часу з використанням таймера, що працює в потоці UI. Це має забезпечити дискретне оновлення станів та безпечну взаємодію з графічним інтерфейсом.

2. При виборі технології графічного інтерфейсу обрати сучасний декларативний фреймворк замість традиційних бібліотек, тому що він надає необхідні інструменти для реалізації 2D-плану (Canvas) та підтримує механізм прив'язки даних.

3. Ядро логіки побудувати на принципах ООП [26]. Створити ієрархію класів пристроїв (від абстрактного базового класу). Їхню взаємодію та автоматизацію (за моделлю ECA) реалізувати через децентралізований поліморфний виклик єдиного методу `UpdateState()`.

4. Зв'язок між бізнес-логікою та візуальним представленням забезпечити через алгоритм прив'язки даних (Binding), що забезпечить перевагу декларативного типу графічного фреймворку.

1.5 Висновки до першого розділу

У цьому розділі було проведено аналіз методів та засобів розробки симуляторів систем управління смарт-будинками. Дослідження ринку існуючих програмних рішень (як комерційних, так і відкритих) дозволило виявити вільну нішу: відсутність легковагових програмних засобів, орієнтованих виключно на

візуальне проєктування та тестування логіки автоматизації, без прив'язки до реального обладнання. Це підтвердило актуальність та сформувало кінцеву ідею програмного проєкту – створення спеціалізованого симулятора.

За результатами аналізу було сформовано загальну ідею реалізації програмного симулятора. Для технічної реалізації цієї ідеї було проведено порівняльний аналіз фундаментальних програмних методів. Цей аналіз дозволив обґрунтувати ключові архітектурні рішення, що стануть основою для проєктування: симуляція з кроком по часу є найбільш придатним «рушієм» для візуальної моделі, децентралізований поліморфний підхід (через абстрактний метод `UpdateState()`) є методом, що повністю задовольняє вимоги ООП та алгоритм прив'язки даних (`Binding`) є найефективнішим способом зв'язку логіки з інтерфейсом. Таким чином, у першому розділі було визначено не лише мету, але й конкретний набір технологій та алгоритмічну базу для подальшої розробки.

Розділ 2. Проєктування архітектури симулятора системи управління смарт-будинком

2.1 Вимоги до симулятора системи управління смарт-будинком

Для кращого розуміння подальшого опису, надано словник ключових термінів проєкту в додатку А.

На основі постановки завдання для програми були окреслені такі основні функціональні і нефункціональні вимоги до симулятора системи управління смарт-будинком:

1. Динамічна симуляція часу: система має мати власний час симуляції, який спливає з прискоренням (6 хвилин в симуляторі вважаються за 1 реальну секунду). Користувач повинен мати можливість зупиняти або запускати та вручну змінювати поточний час симуляції.

2. Візуалізація та інтерфейс: програма має реалізувати двокомпонентний інтерфейс: ліва частина – візуальний 2D-план будинку з інтерактивними областями, права – панель управління та інформації.

3. Інтерактивний план: на статичне зображення плану будинку необхідно накласти інтерактивні напівпрозорі області для приміщень та пристроїв. Області пристроїв повинні мати пріоритет при натисканні над областями приміщень.

4. Відображення станів: області пристроїв мають динамічно змінювати свій колір та колір рамки, відображаючи поточний стан (напр., стани “Працює” – зелена рамка, “Вимкнено” – сіра та ін.).

5. Панель управління: права частина інтерфейсу має надавати користувачу «Інструменти» («Пожежа», «Рухоме тіло», «Молоток» та ін) для впливу на середовище, а також фільтри для ввімкнення або вимкнення видимості груп пристроїв.

6. Контекстна інформація: при натисканні на область приміщення, в нижній правій частині інтерфейсу має з'являтися детальна інформація про це

приміщення: його поточні статуси (температура, вологість, «Пожежа» та ін.) та повний список приладів у ньому з можливістю ручного регулювання їхніх станів.

7. Логіка пристроїв: необхідно реалізувати детальну та унікальну логіку для кожного типа пристроїв, включаючи клімат-контроль (реакція приладів на зміну температури та вологості в приміщенні), безпеку (реакція камер та сигналізації на «Рух», «Злом»), пожежну систему з автономною ймовірністю спрацювання кожного приладу водяного пожежогасіння та освітлення (ручне та автоматичне).

8. Система електроживлення: має бути реалізована глобальна система електроенергії (кнопка «Увімкнути електроенергію» або «Вимкнути електроенергію»). Поведінка пристроїв має залежати від наявності живлення, з урахуванням резервного живлення від сонячних панелей та батареї для критичних систем (камери, сигналізація).

9. Логування: усі значущі події (зміна стану пристрою, спрацювання сирени, дії користувача, зміна часу тощо) мають автоматично записуватися у текстовий лог з фіксацією часу симуляції.

2.2 Проектування структури симулятора системи управління смарт-будинком

Реалізація вимог до симулятора управління смарт-будинком та забезпечення гнучкості його програмної архітектури вимагає використання багат шарового архітектурного підходу. Це передбачає розділення системи на логічні рівні, кожен з яких відповідає за окремий аспект функціонування, що спрощує масштабування та підтримку коду.

2.2.1 Компоненти, які включає в себе симулятор системи управління смарт-будинком

До складу симулятора системи управління смарт-будинком входять такі модулі:

1. Модуль «Графічний Інтерфейс», що відповідає за візуалізацію, містить полотно для відмальовування плану. Це контейнер, який дозволяє розміщувати графічні примітиви за абсолютними координатами. Також модуль має панель інструментів із функціоналом для вибору режимів зовнішнього впливу (наприклад, «Вогонь», «Рух») і налаштування фільтрів видимості та інформаційну панель, яка динамічно оновлює свій вміст і відображає детальні параметри залежно від обраного об'єкту.

2. Модуль «Центральний Контролер» виступає ядром системи, яке керує роботою всіх інших компонентів, починаючи з ініціалізації процесу симуляції та завантаження початкової конфігурації областей приміщення і пристроїв. Модуль містить вбудований таймер симуляції, що генерує тактові імпульси з заданою частотою для синхронізації процесів, а також відповідає за перехоплення входних подій від інтерфейсу (кліки, натискання кнопок) і їх трансляцію у відповідні команди для оновлення моделі.

3. Модуль «Модель Даних» слугує централізованим сховищем поточного стану всієї системи, структуруючи дані на кількох ієрархічних рівнях. Він описує глобальний стан зі змінними на кшталт поточного часу та статусу електромережі, колекцію[27] об'єктів областей приміщення із їхніми параметрами середовища, а також поліморфну колекцію всіх смарт-пристроїв. Це забезпечує збереження цілісності даних та доступ до актуальних параметрів кожного елемента інфраструктури будинку.

4. Модуль «Бізнес-логіка Пристроїв» є сукупністю правил та поведінкових алгоритмів, що інкапсульовані безпосередньо всередині класів пристроїв. Завдяки такому підходу кожен пристрій володіє «знаннями» про власну специфіку роботи та автономно реагує на зміну часу, зміни параметрів середовища або зовнішні події відповідно до закладеної логіки.

2.2.2 Потік даних всередині симулятора системи управління смарт-будинком

Взаємодія між модулями відбувається циклічно:

- 1) тік таймера: «Центральний Контролер» отримує сигнал від таймера про проходження чергового періоду часу;
- 2) оновлення середовища: «Центральний Контролер» оновлює час симуляції та розраховує зміни параметрів середовища (наприклад, зміну температури в областях приміщень);
- 3) оновлення пристроїв: «Центральний Контролер» проходить по списку всіх пристроїв і викликає їхній метод оновлення стану;
- 4) реакція пристроїв: кожен пристрій аналізує свій стан та стан своєї області приміщення і, за необхідності, змінює свій статус (наприклад, обігрівач вмикається);
- 5) оновлення інтерфейсу: «Графічний Інтерфейс» автоматично (через механізм прив'язки даних) відображає нові стани пристроїв на екрані.

2.3. Проектування об'єктно-орієнтованої моделі симулятора системи управління смарт-будинком

Основою гнучкості коду симулятора системи управління смарт-будинком є правильно спроектована ієрархія класів[28]. Використання принципів ООП (інкапсуляція, наслідування, поліморфізм) дозволить уніфікувати роботу з різними типами обладнання.

2.3.1 Базовий абстрактний клас

Всі пристрої в системі будуть нащадками єдиного абстрактного класу SmartDeviceBase. Це гарантує, що будь-який об'єкт на плані матиме спільний інтерфейс взаємодії.

1. Атрибути класу SmartDeviceBase:
 - 1) Name (назва) – унікальне ім'я пристрою (напр., «Лампа Л1»);
 - 2) Location (розташування) – посилання на об'єкт області приміщення, де встановлено пристрій;
 - 3) Position (координати) – X та Y координати для відображення на плані;

4) CurrentState (стан) – перелік поточного стану («Працює», «Вимкнено», «Зламано» тощо).

2. Методи класу SmartDeviceBase:

1) UpdateState() – абстрактний метод, головна точка входу для логіки автоматизації, кожен нащадок повинен реалізувати свою власну версію цього методу;

2) Interact() – віртуальний метод для обробки прямої взаємодії з користувачем (клік мишкою).

2.3.2 Спеціалізовані групи та класи

Ієрархія класів наслідуваних від єдиного абстрактного класу SmartDeviceBase розгалужується на функціональні групи:

1. ClimateControlUnit («Кліматична техніка») – це абстрактний клас-нащадок для приладів, що впливають на атмосферу. Містить конкретні класи: HeaterDevice («Обігрівач»), ConditionerDevice («Кондиціонер»), HumidifierDevice («Зволожувач»), DehumidifierDevice («Осушувач»). За логікою кліматичної техніки в методі UpdateState прилади шукають у своїй області приміщення термостат і порівнюють поточну температуру або вологість з бажаною.

2. Прилади та об'єкти, що напряду наслідуються від єдиного абстрактного класу SmartDeviceBase:

А. Група сенсорів та контролерів представлена класами ThermostatDevice («Термостат»), WindowDevice («Вікно») та DoorDevice («Двері»). Клас ThermostatDevice зберігає цільові значення температури та вологості, які встановлює користувач для конкретного приміщення, і слугує орієнтиром для кліматичної техніки. Класи WindowDevice та DoorDevice, своєю чергою, моделюють стан фізичних об'єктів («Відкрито»/«Закрито») або їхню цілісність («Ціле»/«Зруйноване»). Саме зміна стану цих об'єктів (наприклад, руйнування вікна) є тригером, що запускає ланцюгову реакцію в системі безпеки.

В. Системи безпеки та освітлення представлені класами MotionSensorLamp («Лампа»), CameraDevice («Камера») та SirenDevice («Сирена»). Клас MotionSensorLamp має внутрішній таймер: при виявленні руху в області приміщення лампа автоматично вмикається і запускає зворотний відлік до вимкнення. Аналогічну логіку має клас CameraDevice, який, однак, має вищий пріоритет енергоспоживання та живиться від резервного джерела при знеструмленні. Окремим елементом є клас SirenDevice, яка не має власних сенсорів, а активується за командою інших пристроїв (датчиків злomu) і залишається в активному стані протягом заданого часу стану Тривоги.

С. Група систем пожежогасіння реалізована через клас FireSprinklerDevice («Водяний спринклер»), який демонструє складну поведінку. Цей пристрій може перебувати в одному з декількох станів: «Готовий» (очікування), «Активний» (процес гасіння), «Пустий» (вичерпання запасу води) або «Перезарядка». Важливою особливістю цього класу є вбудований параметр ймовірності, який додає елемент випадковості у процес спрацювання системи при виявленні пожежі, наближаючи симуляцію до реальних умов.

Д. Група енергетичної інфраструктура забезпечує життєдіяльність будинку і представлена класами BatteryDevice («Батарея») та SolarPanelDevice («Сонячна панель»). Клас BatteryDevice моделює процес накопичення енергії при наявності зовнішньої мережі та її віддачу споживачам при знеструмленні. Клас SolarPanelDevice реалізує залежність від часу доби, генеруючи електроенергію лише у визначений світловий проміжок (наприклад, з 8:00 до 16:00), що вимагає від «Центрального Контролера» враховувати поточний час симуляції при розрахунку енергобалансу.

Для розуміння усіх областей приладів та об'єктів з їх функціями було створено список наведений у додатку Б.

2.4 Опис основних алгоритмів функціонування системи

Для забезпечення коректної роботи симулятора розроблено ряд алгоритмів, що керують різними аспектами його моделі:

1. Алгоритм головного циклу симуляції (рис. 2.1) є основним механізмом, що забезпечує життєдіяльність системи. Він запускається таймером з фіксованою частотою. На початку кожного такту алгоритм перевіряє статус паузи; якщо симуляція активна, відбувається збільшення поточного часу симуляції. Далі виконується аналіз енергетичної системи – перевіряється наявність зовнішнього живлення, генерація від SolarPanelDevice та рівень заряду BatteryDevice, на основі чого формується статус наявності енергії в будинку. Після цього алгоритм перераховує параметри середовища (температуру та вологість) для кожної області приміщення. Наступним кроком є послідовний виклик методу оновлення станів для кожного пристрою в системі. Завершується цикл оновленням «Графічного Інтерфейсу» для відображення актуального стану моделі.

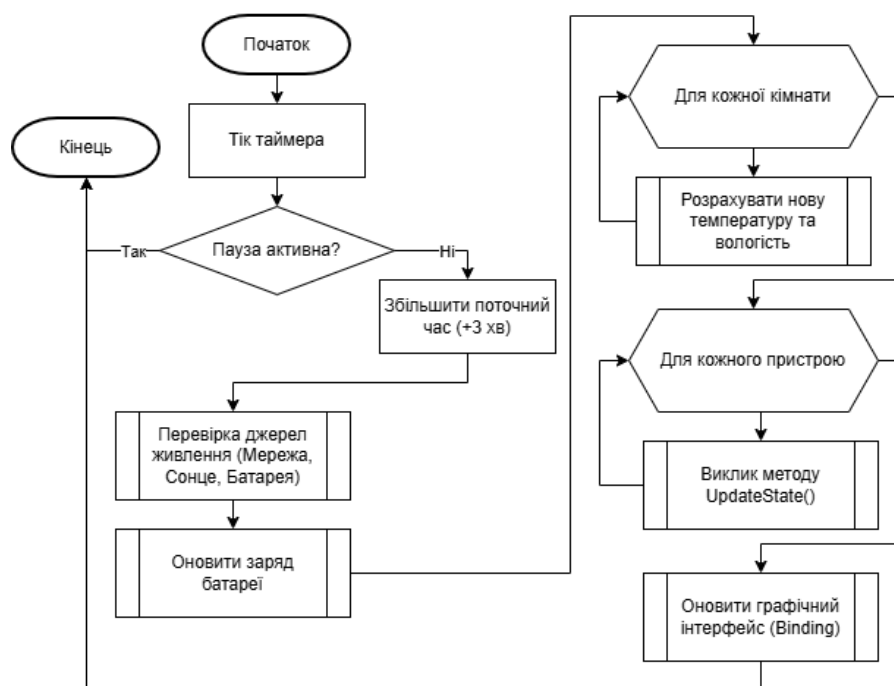


Рисунок 2.1. Блок-схема «Алгоритм головного циклу симуляції»

2. Алгоритм роботи інструменту «Рухоме тіло» (рис. 2.2) описує реакцію системи на дії користувача. При виборі інструменту і кліку на область приміщення система ідентифікує її об'єкт та встановлює в ньому прапорець наявності руху. Одночасно розраховується час завершення цієї події. При

наступному проходженні головного циклу симуляції всі пристрої в цій області приміщення, що мають відповідні сенсори (наприклад, CameraDevice, MotionSensorLamp), зчитують цей прапорець у своєму методі оновлення станів і автоматично перейдуть в активний режим роботи.

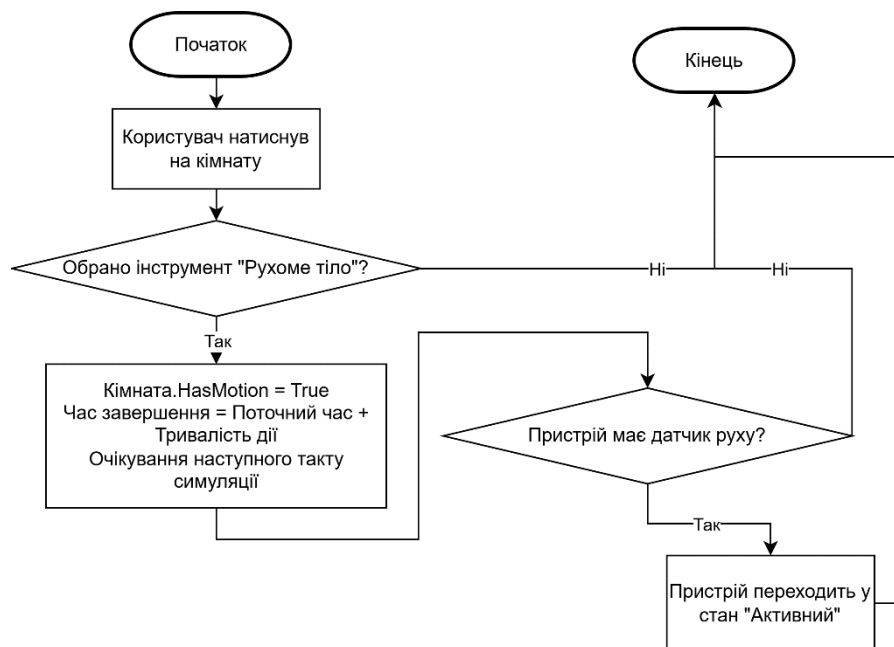


Рисунок 2.2. Блок-схема «Алгоритм інструменту «Рухоме тіло»»

3. Алгоритм роботи інструменту «Пожежа» (рис. 2.3) моделює виникнення аварійної ситуації. Якщо користувач обирає цей інструмент та натискає на певну область приміщення, система змінює внутрішній стан відповідного об'єкта області приміщення, встановлюючи прапорець наявності вогню. Ця подія фіксується в системному журналі. Надалі, пристрої класу FireSprinklerDevice, що знаходяться в цій області приміщення, під час свого циклу оновлення виявляють зміну статусу і, відповідно до закладеної ймовірності, можуть активувати режим гасіння.

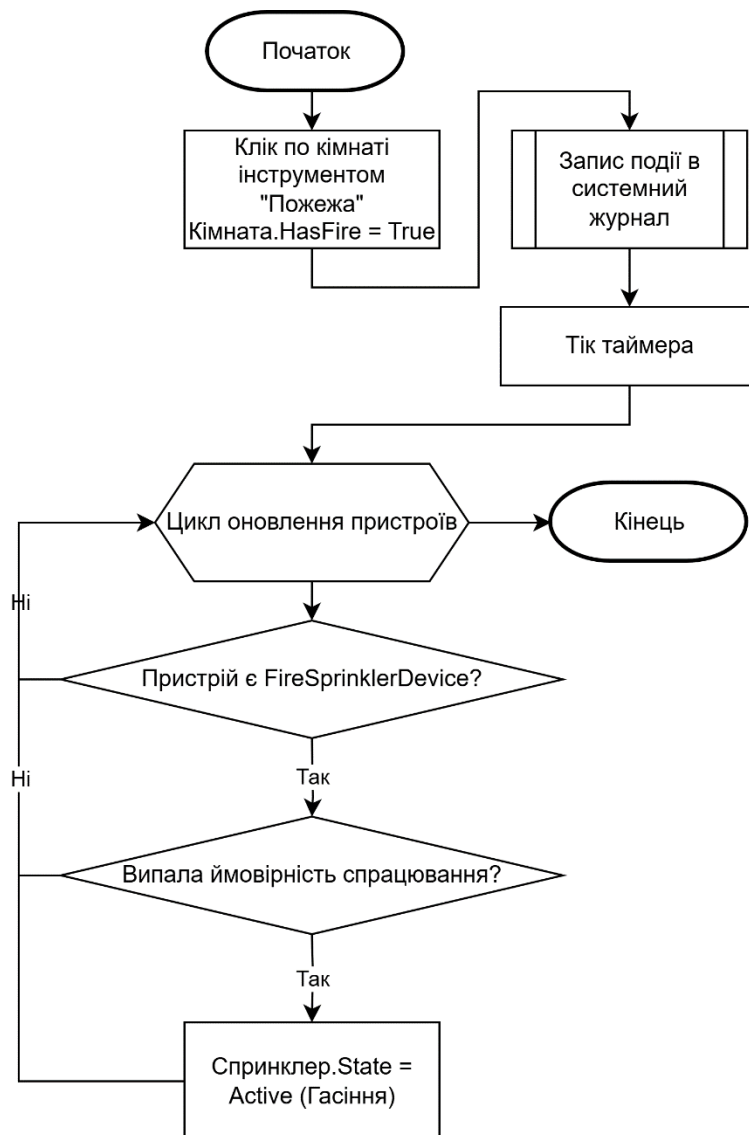


Рисунок 2.3. Блок-схема «Алгоритм інструменту «Пожежа»»

4. Алгоритм роботи інструменту «Злом» (Молоток) (рис. 2.4) описує взаємодію з інфраструктурними об'єктами. Застосування інструменту дозволено лише для пристроїв класів WindowDevice та DoorDevice. При кліку на такий об'єкт його стан примусово змінюється на «Зруйновано», а область приміщення отримує статус «Взлом». Система миттєво реагує на цю подію – «Центральний Контролер» перевіряє наявність пристрою SirenDevice і, якщо такий існує, надсилає йому команду на активацію, що запускає режим тривоги та відповідний звуковий сигнал.

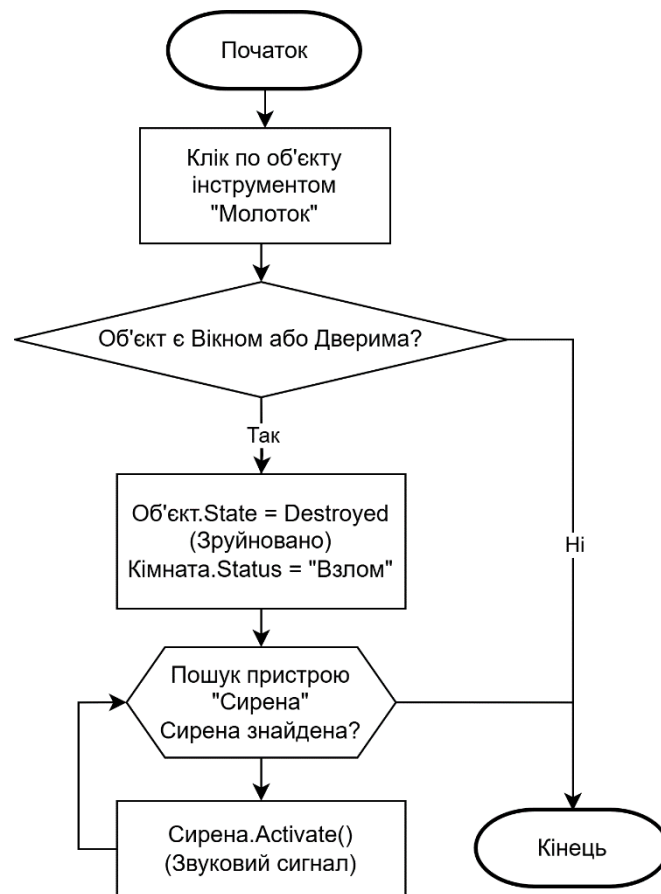


Рисунок 2.4. Блок-схема «Алгоритм інструменту «Злом»»

5. Алгоритм роботи Спринклера (Система пожежогасіння) (рис. 2.5) демонструє складну логіку станів. Під час оновлення пристрій FireSprinklerDevice перевіряє поточний час та свій стан. Якщо він знаходиться в режимі перезарядки і час вийшов – переходить у стан готовності. Якщо він активний (гасить пожежу) і вичерпав ресурс часу – переходить у стан «Пустий». У стані готовності пристрій перевіряє наявність вогню в області приміщення та електроживлення. Якщо умови виконуються, генерується випадкове число, і якщо воно відповідає заданій ймовірності, спринклер активується, починаючи процес гасіння.

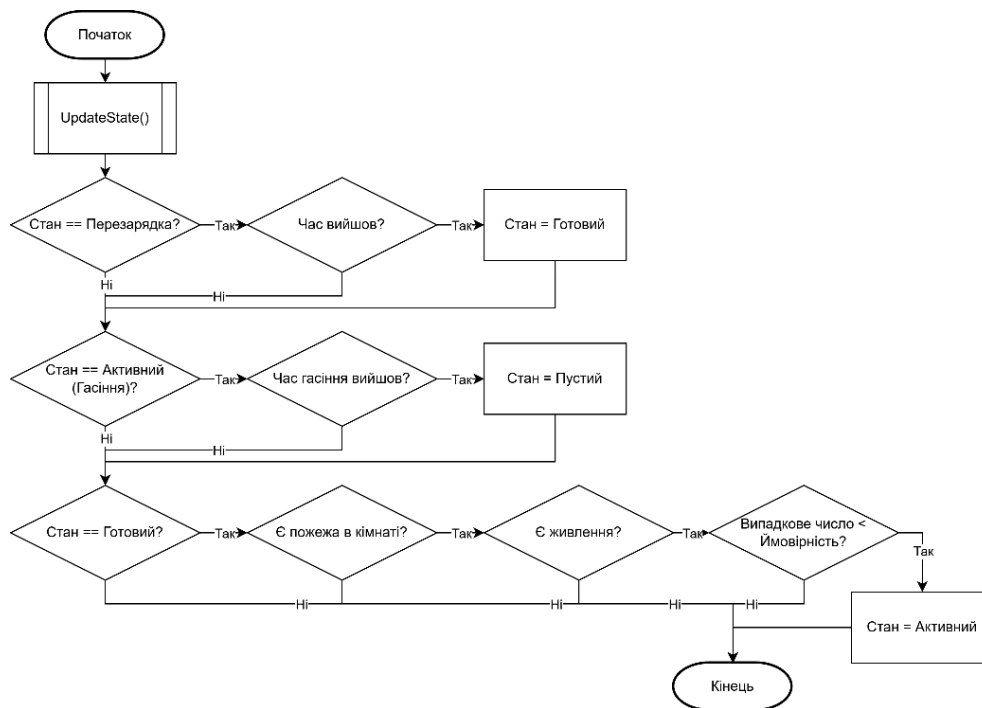


Рисунок 2.5. Блок-схема «Алгоритм роботи Спринклера (Машина станів)»

6. Алгоритм роботи системи клімат-контролю (рис. 2.6) полягає в тому, що кліматичний пристрій (наприклад, HeaterDevice або DehumidifierDevice) реалізує принцип зворотного зв'язку. При виклику методу оновлення станів пристрій спершу перевіряє наявність електроживлення. Якщо живлення відсутнє, пристрій переходить у вимкнений стан. Якщо живлення є, пристрій звертається до ThermostatDevice, розташованого в тій же області приміщення, і отримує бажане користувачем значення температури або вологості. Далі він зчитує ті самі поточні параметри області приміщення. Шляхом порівняння цих двох значень пристрій приймає рішення: якщо поточна температура нижча за цільову, HeaterDevice активується, в іншому випадку – переходить у режим очікування і якщо поточна вологість вища за цільову, DehumidifierDevice активується, в іншому випадку – переходить у режим очікування.

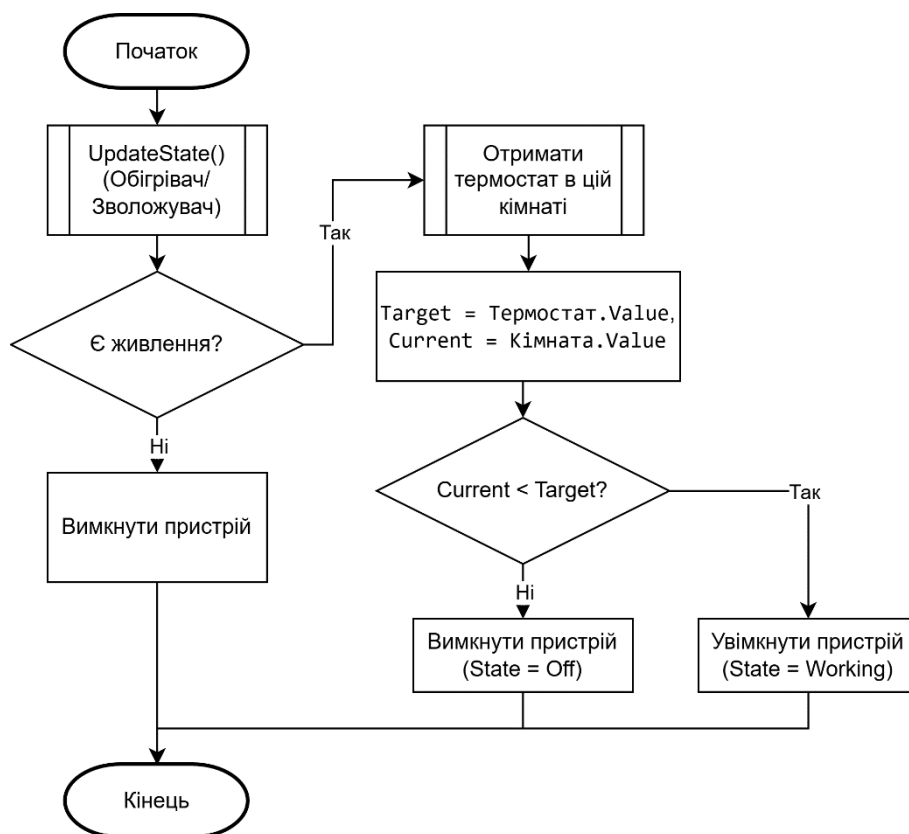


Рисунок 2.6. «Блок-схема Алгоритм клімат-контролю»

Далі представлена узагальнена структурна схема симулятора (рис. 2.7).



Рисунок 2.7. Узагальнена структурна схема симулятора системи управління смарт-будинком

2.5 Висновок до другого розділу

У даному розділі виконано комплексне проєктування програмного симулятора. Сформульовано функціональні вимоги, що охоплюють всі аспекти

роботи системи. Розроблено модульну архітектуру, що розділяє відповідальність між інтерфейсом, логікою та даними. Спроектовано ієрархію класів, яка дозволяє гнучко моделювати поведінку різноманітних пристроїв. Описані алгоритми окреслюють логіку взаємодії компонентів та реакцію системи на події. Результати проєктування створюють базу для етапу програмної реалізації.

Розділ 3. Програмна реалізація та тестування симулятора системи управління смарт-будинком

3.1 Обґрунтування вибору засобів розробки

Для програмної реалізації симулятора було обрано мову програмування C# та платформу .NET [29]. Цей вибір зумовлений суворою типізацією мови, розвиненими можливостями для об'єктно-орієнтованого програмування та наявністю потужних бібліотек LINQ [30] для роботи з колекціями даних, що є критично важливим для обробки списків пристроїв.

В якості технології для побудови графічного інтерфейсу обрано Windows Presentation Foundation (WPF). На відміну від застарілих технологій (WinForms), WPF надає наступні переваги для даного проєкту:

- використання векторної графіки та Canvas, контейнери якого дозволили розміщувати об'єкти пристроїв поверх плану будинку за абсолютними координатами X та Y, що забезпечує точність візуалізації;
- чітке розділення логіки та представлення засобами XAML, де інтерфейс описано декларативною мовою розмітки XAML (MainWindow.xaml), тоді як логіка роботи зосереджена в C# коді (MainWindow.xaml.cs);
- застосування ключового механізму прив'язки даних Binding, що дозволив зв'язати візуальні властивості об'єктів (колір рамки, видимість) безпосередньо з логічними властивостями класів (CurrentState, IsVisibleByUser), завдяки чому та використанню інтерфейсу INotifyPropertyChanged [31] будь-яка зміна стану в коді миттєво відображається на екрані без необхідності писати код для ручного перемальовування.

Середовищем розробки обрано Microsoft Visual Studio [32], яке надає зручні інструменти для відлагодження коду, дизайнер XAML [33] та засоби рефакторингу.

3.2 Розробка компонентів симулятора

Структура реалізованого додатка відповідає спроектованій у Розділі 2 архітектурі. Повний текст коду розроблених програмних компонентів розміщено в додатку В. Також, у додатку Г наведено повні діаграми класів симулятора, що розробляється.

Головний клас вікна `MainWindow` виконує роль «Центрального контролера». Для управління часом симуляції реалізовано таймер `_simulationTimer` типу `DispatcherTimer` [34]. Він налаштований на спрацювання кожні 0.5 секунди (константа `UPDATE_INTERVAL_SECONDS`), що відповідає одному «такту» симуляції. У кожному такті системний час збільшується пропорційно коефіцієнту прискорення.

```
private void InitializeSimulation()
{
    CurrentSimTime = TimeSpan.Zero;
    _simulationTimer = new DispatcherTimer();
    _simulationTimer.Interval = TimeSpan.FromSeconds(UPDATE_INTERVAL_SECONDS);
    _simulationTimer.Tick += SimulationTimer_Tick; // Підписка на подію такту
    // ... ініціалізація таймера тривоги
}
```

Зберігання даних реалізовано через типізовані колекції:

- `_allRooms (List<Room>)` – містить об'єкти областей приміщення з їхніми параметрами;
- `_allDevices (List<SmartDeviceBase>)` – містить повний список усіх пристроїв;
- `DevicesOnPlan (ObservableCollection<DeviceViewModel>)` – спеціальна колекція для прив'язки до інтерфейсу, яка забезпечує відображення пристроїв на Canvas.

Для візуалізації станів пристроїв розроблено конвертери значень (`DeviceBorderColorConverter`, `DeviceFillColorConverter`), які трансформують стан

DeviceState (наприклад, Working, Destroyed) у відповідні кольори (зелений, чорний тощо) безпосередньо в XAML-розмітці.

3.3 Застосування наслідування, інкапсуляції та поліморфізму в проєктуванні симулятора системи управління смарт-будинком

Інкапсуляція застосована для захисту внутрішнього стану об'єктів. Властивість CurrentState у базовому класі має захищений сетер (protected set), що дозволяє змінювати стан лише самому пристрою або його нащадкам, але не зовнішнім класам напрямую.

Наслідування реалізовано через створення абстрактного базового класу SmartDeviceBase, який містить спільні властивості (Name, Position, AssociatedRoom) та методи. Всі конкретні пристрої (HeaterDevice, CameraDevice тощо) наслідуються від нього.

```
public abstract class SmartDeviceBase : INotifyPropertyChanged
{
    public string Name { get; }
    public DeviceState CurrentState { get; protected set; }
    // ... інші властивості

    // Абстрактний метод, який змушує нащадків реалізувати власну логіку
    public abstract void UpdateState(TimeSpan currentTime, bool isElectricityOn,
Room environment);
}
```

Поліморфізм реалізується через «Центральний Контролер», який зберігає всі пристрої в одному списку типу List<SmartDeviceBase>. У циклі оновлення він викликає метод device.UpdateState() для кожного елемента. Завдяки поліморфізму, для кожного пристрою виконується саме його унікальний код, хоча виклик виглядає однаково.

Для об'єднання різномірних пристроїв за спільною поведінкою будуть використані інтерфейси. Наприклад, інтерфейс IOnOffToggleable («Той, що перемикається») буде реалізований у класах Люстри, Вентилятора та Плити. Це

дозволить обробляти клік мишкою на будь-якому з цих пристроїв одним універсальним методом ToggleState().

Приклад поліморфної реалізації для HeaterDevice (пристрій «Обігрівач»), який реалізує логіку клімат-контролю:

```
public class HeaterDevice : ClimateControlUnit
{
    public override void UpdateState(TimeSpan currentTime, bool isElectricityOn,
    Room environment)
    {
        if(!isElectricityOn) { if(CurrentState != DeviceState.Off) TurnOff();
return; }

        // Пошук термостата в тій же області приміщення
        var thermostat = environment?.Devices.OfType<ThermostatDevice>()
            .FirstOrDefault(t => t.CurrentState ==
DeviceState.Working);

        if(thermostat != null)
        {
            // Порівняння поточної температури з цільовою
            if(environment.CurrentTemperature < thermostat.TargetTemperature)
                TurnOn(isElectricityOn);
            else if(environment.CurrentTemperature >=
thermostat.TargetTemperature)
                TurnOff();
        }
    }
}
```

3.4 Тестування симулятора системи управління смарт-будинком

Метою тестування була перевірка якості реалізації функцій симулятора системи управління смарт-будинком. Застосовано методику тестування симулятора в цілому. В якості засобів тестування використано типові сценарії функціонування системи управління смарт-будинком. Результати тестування представлені нижче.

Сценарій 1. Початковий стан системи – за стандартними налаштуваннями симулятора системи управління смарт-будинком (рис. 3.1). Перевіряється реакція системи на подію «Пожежа».

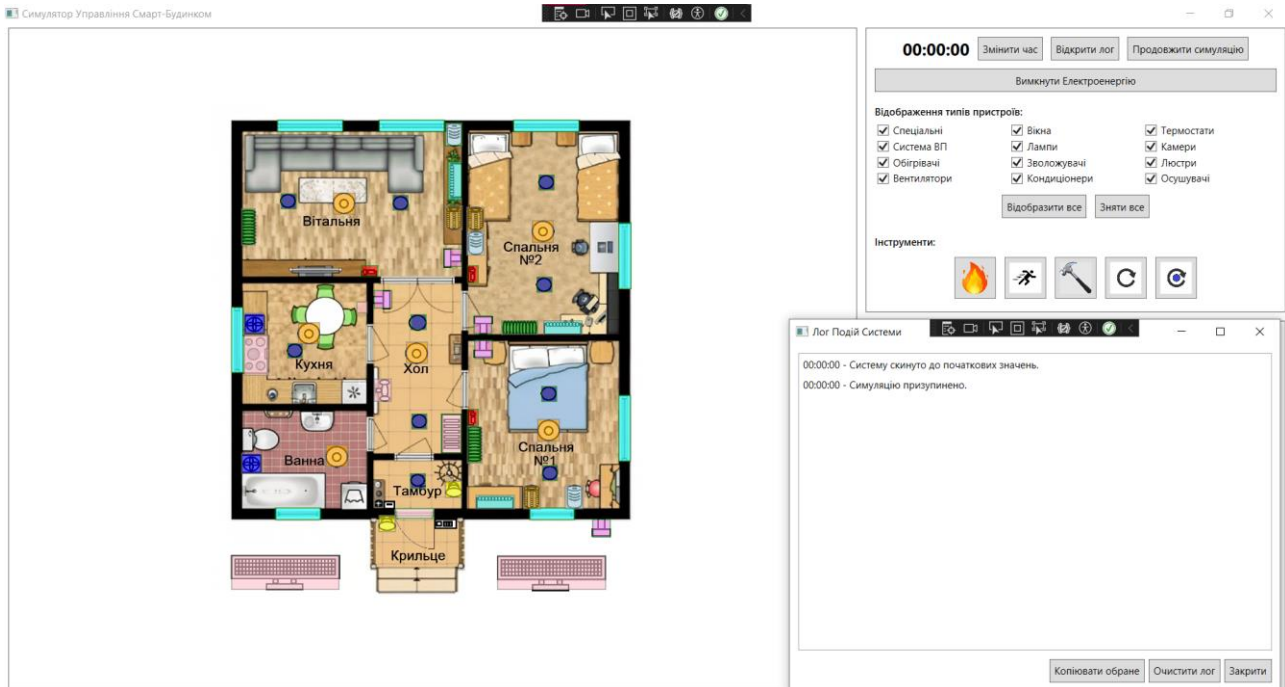


Рисунок 3.1. Стандартні налаштування симулятора системи управління смарт-будинком з вікном «Лог Подій Системи»

При використанні інструменту в області приміщення, в описі якої з'явився статус «Пожежа» (рис. 3.2).

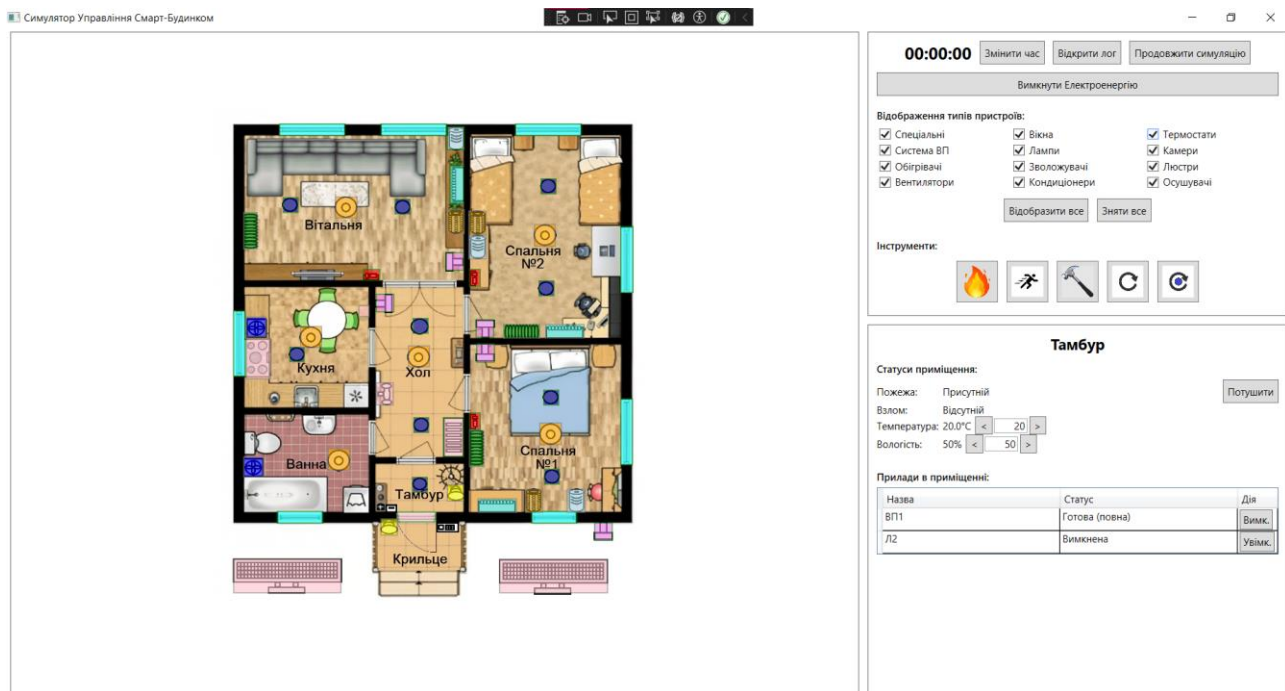


Рисунок 3.2. Відображення статусу «Пожежа» в області приміщення «Тамбур»

Кожні 3 хв симуляції має з певною ймовірністю активуватися спринклер «Водяне Пожежогасіння», і на заданому плані будинку, в області приміщення в якій розташований, змінити його відображення (рис. 3.3).

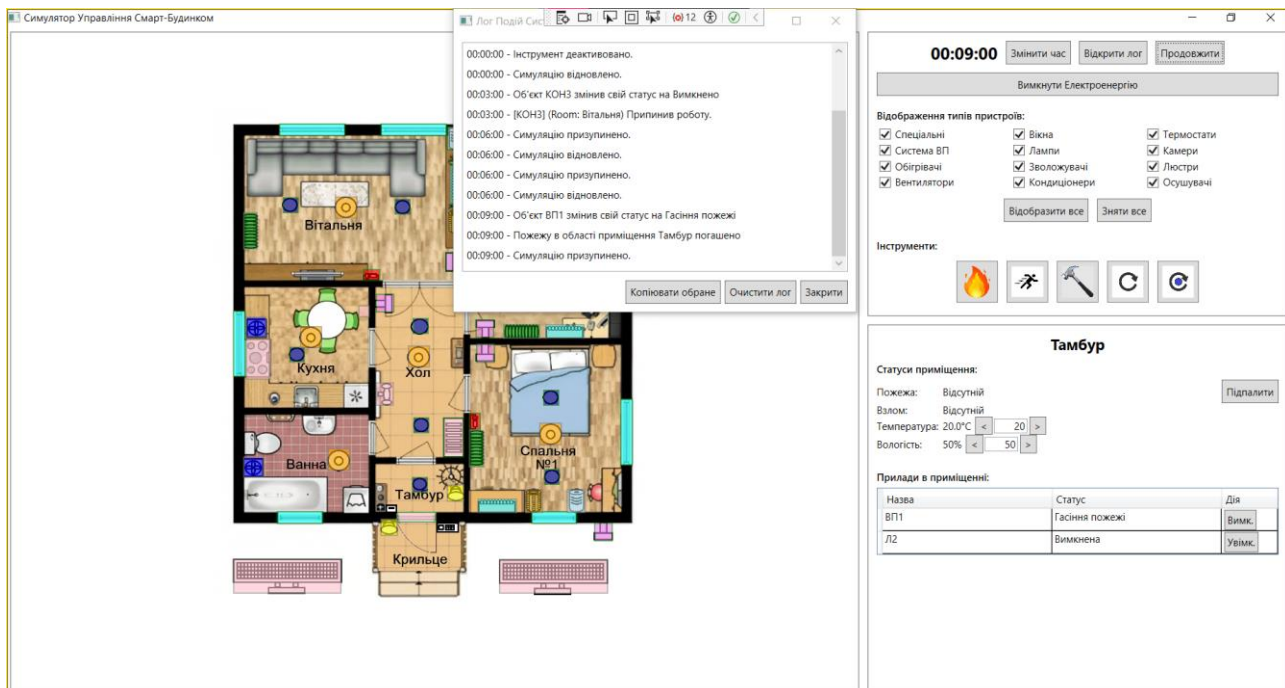


Рисунок 3.3. Відображення поточного стану спринклера в області приміщення «Тамбур» («Гасіння пожежі») з вікном «Лог Подій Системи»

Сценарій 2. Перевірка логіки роботи системи пожежогасіння (спринклера).

Сценарій є продовженням події «Пожежа». Спринклер в області приміщення зі статусом пожежі, враховуючи наявність живлення та ймовірність спрацювання, переходить у стан «Гасіння пожежі», змінюючи колір рамки на зелений. Після завершення циклу гасіння спринклер переходить у стан «Пустий», а статус пожежі знімається (рис. 3.4).

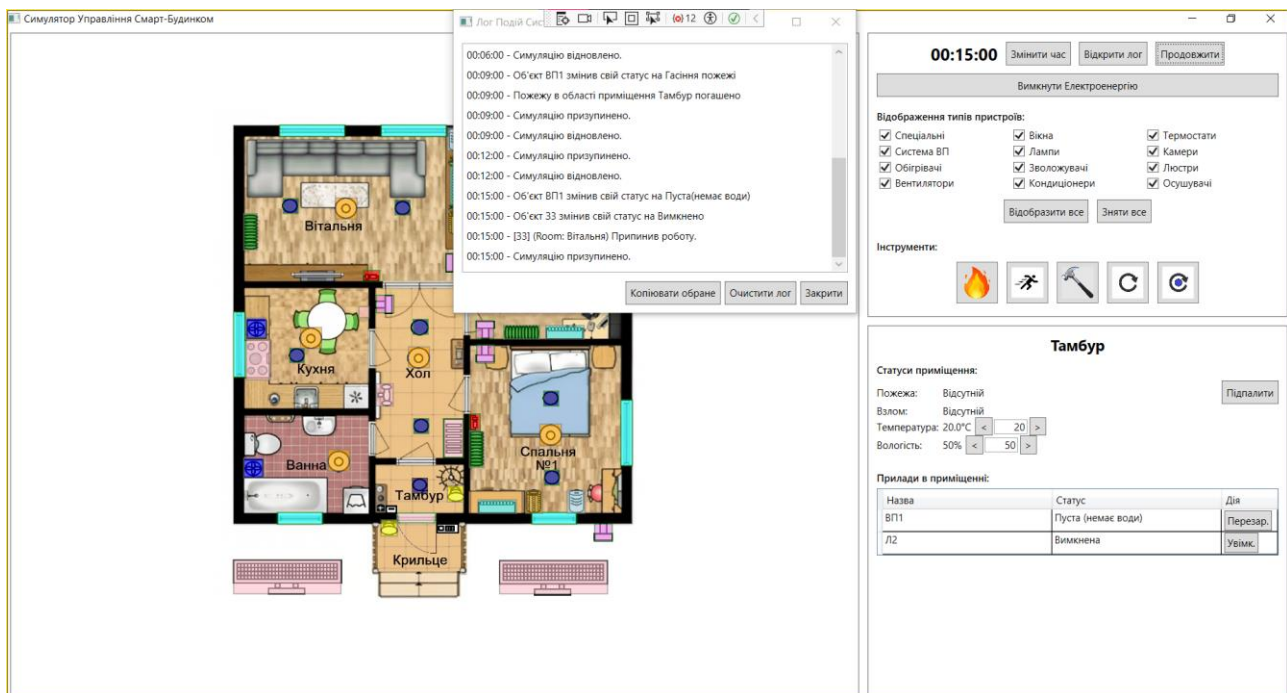


Рисунок 3.4. Відображення поточного стану спринклера в області приміщення «Тамбур» («Пустий (немає води)») з вікном «Лог Подій Системи»

Через визначений час спринклер автоматично розпочинає процес перезарядки, переходячи у стан «Перезарядка», що демонструє повний цикл роботи машини станів пристрою (рис. 3.5).

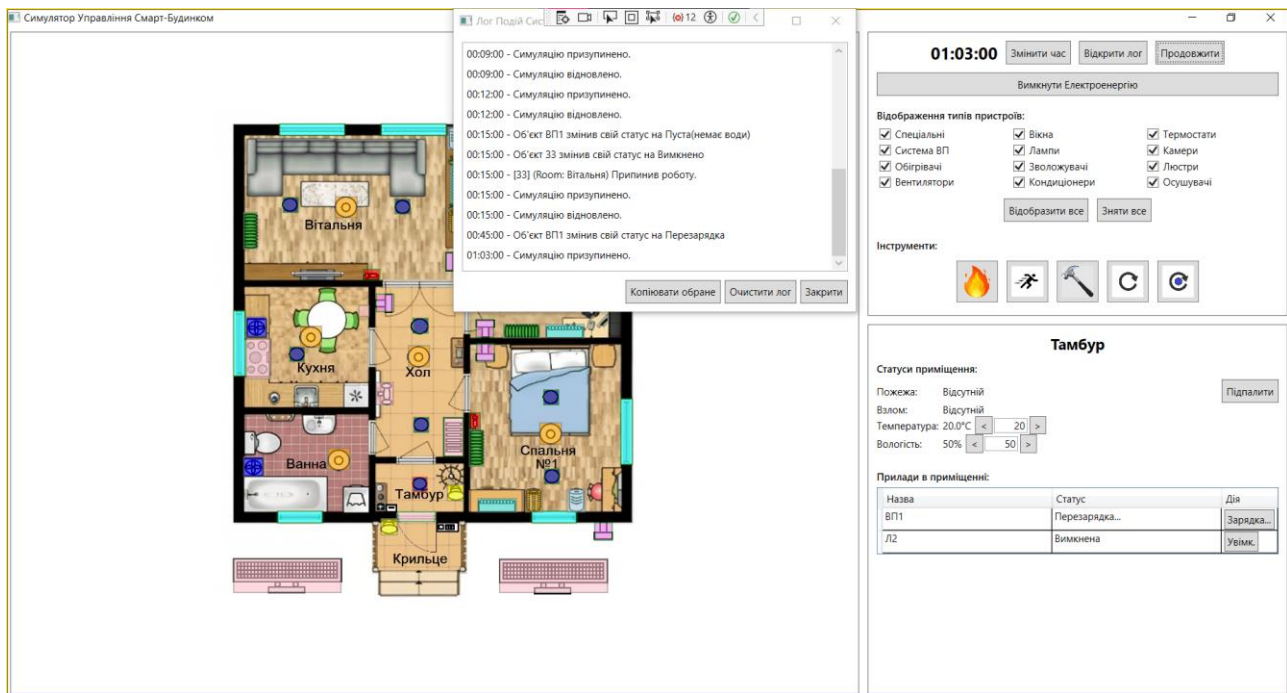


Рисунок 3.5. Відображення поточного стану спринклера в області приміщення «Тамбур» («Перезарядка») з вікном «Лог Подій Системи»

Сценарій 3. Перевірка алгоритму головного циклу симуляції та роботи енергосистеми. Початковий стан системи – стандартні налаштування, час 12:00, відсутня генерація від сонячних панелей (рис. 3.6).

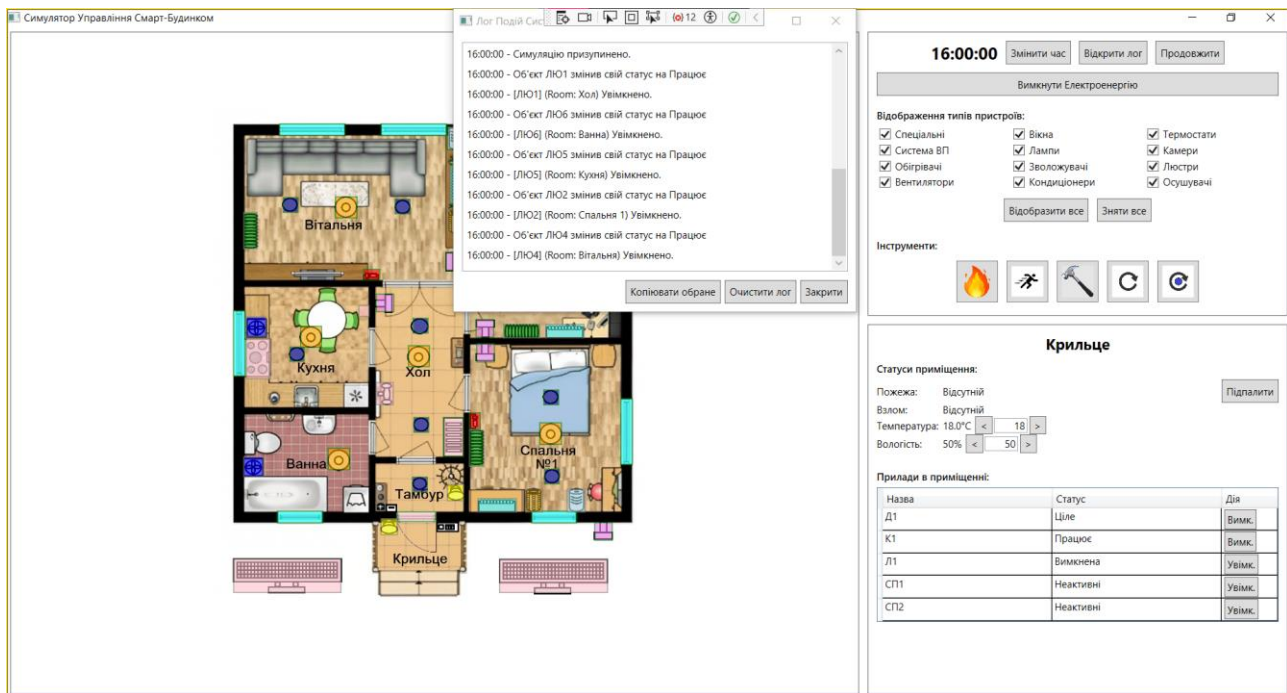


Рисунок 3.6. Відображення поточного стану сонячних панелей в області приміщення «Крильце» («Неактивні») з вікном «Лог Подій Системи»

Користувач натискає кнопку «Вимкнути електроенергію». Система переходить у режим живлення від батареї, статус якої змінюється на «Розрядження». При цьому критичні системи продовжують працювати, а некритичні (наприклад, обігрівачі чи люстри) автоматично вимикаються, що підтверджує коректність роботи логіки пріоритетів живлення та оновлення станів у головному циклі (рис. 3.7).

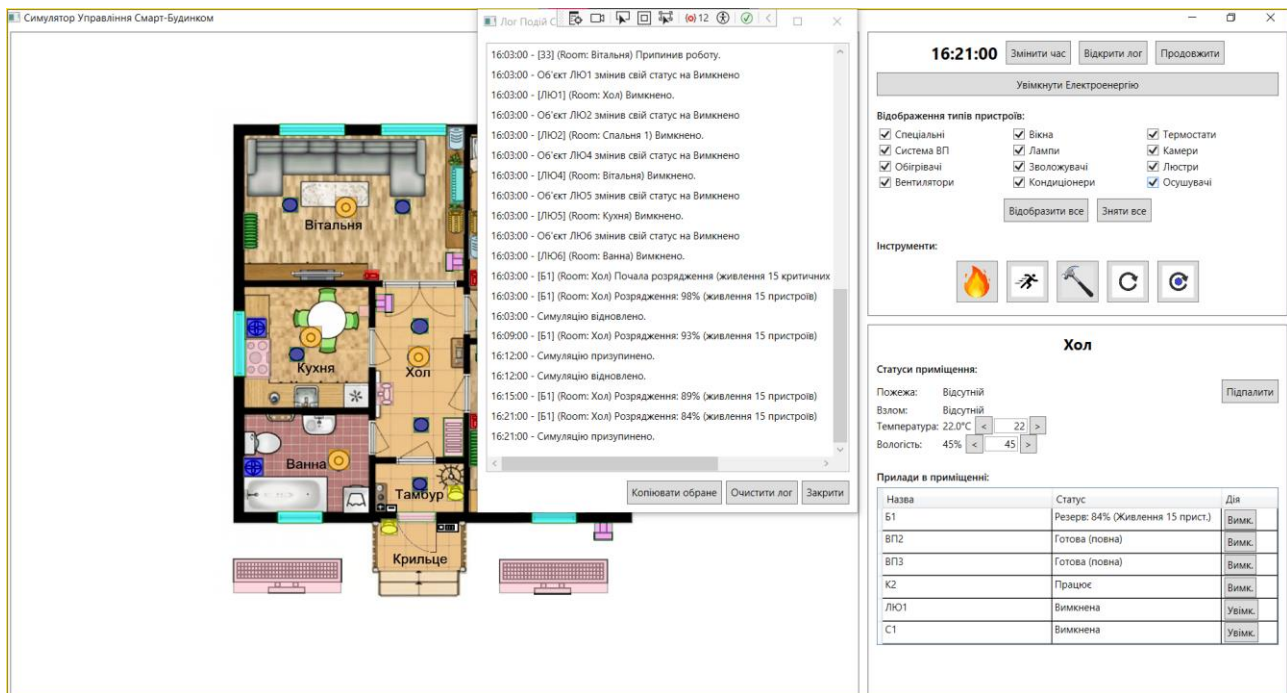


Рисунок 3.7. Відображення роботи симулятора в стані «Без електроенергії» з вікном «Лог Подій Системи»

Сценарій 4. Перевірка реакції на інструмент «Рухоме тіло». Початковий стан – система в режимі очікування, пристрої безпеки активні. Користувач обирає інструмент «Рухоме тіло» та натискає на область приміщення «Крильце». Система встановлює прапорець наявності руху в цій області приміщення, після чого камера відеоспостереження CameraDevice автоматично переходить у режим запису, а лампа MotionSensorLamp вмикається (рис. 3.8).

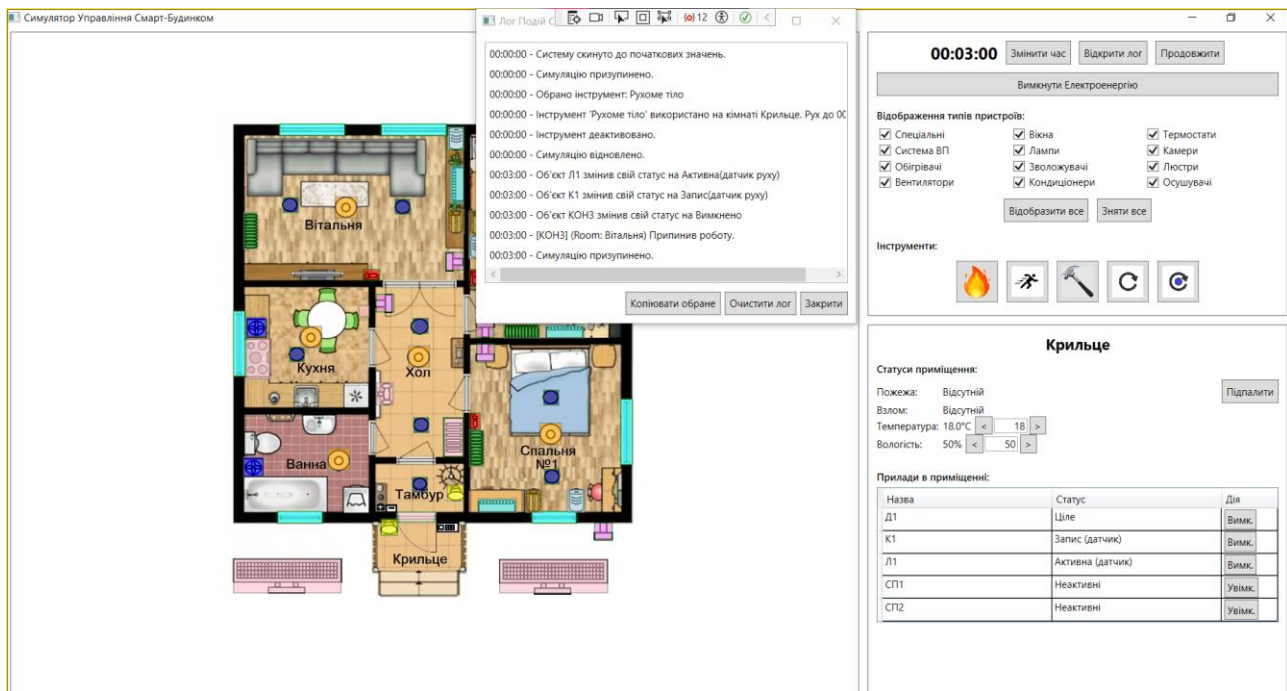


Рисунок 3.8. Відображення поточного стану камери та лампи в області приміщення «Крильце» з вікном «Лог Подій Системи»

Через заданий проміжок часу (30 хвилин часу симуляції) пристрої автоматично повертаються у стан очікування, що підтверджує правильну роботу таймерів подій (рис. 3.9).

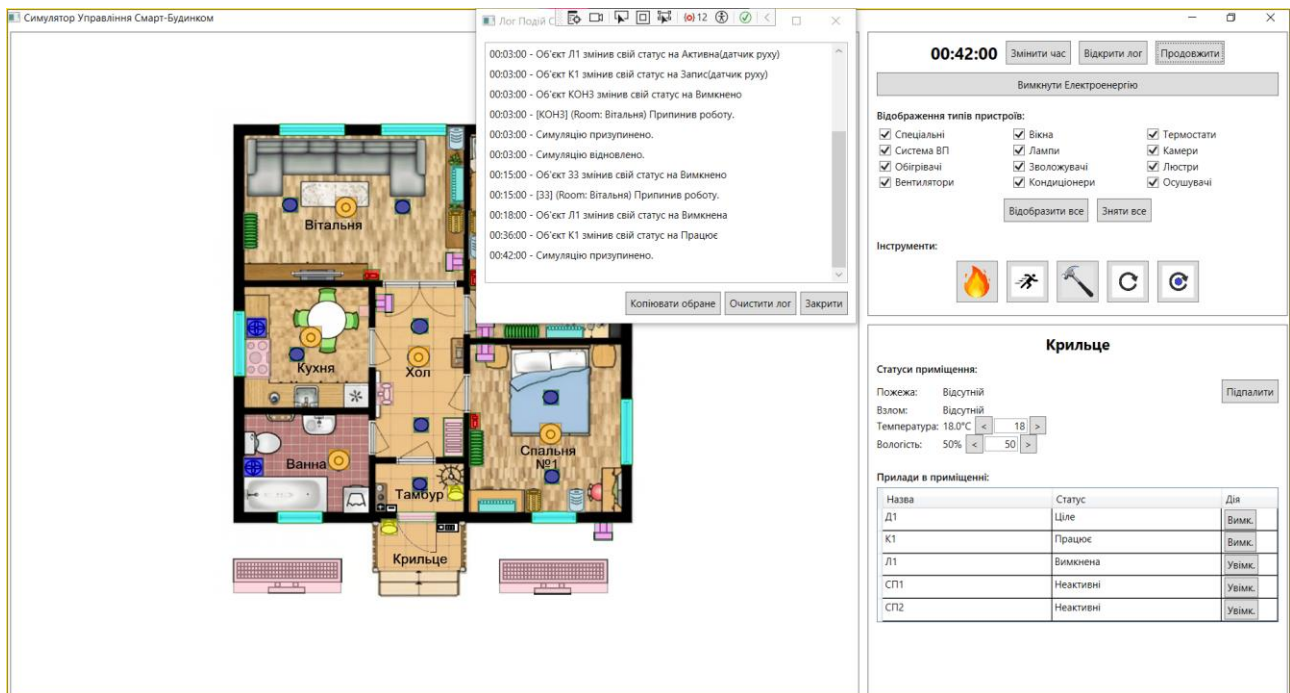


Рисунок 3.9. Відображення поточного стану камери та лампи в області приміщення «Крильце» (зупинили запис) з вікном «Лог Подій Системи»

Сценарій 5. Перевірка реакції на подію «Злом» та роботу сирени. Початковий стан – всі вікна та двері цілі. Користувач застосовує інструмент «Молоток» до об'єкта «Вікно В1». Стан вікна миттєво змінюється на «Зруйновано», його відображення на плані стає сірим, а область приміщення отримує статус «Взлом» (рис. 3.10).

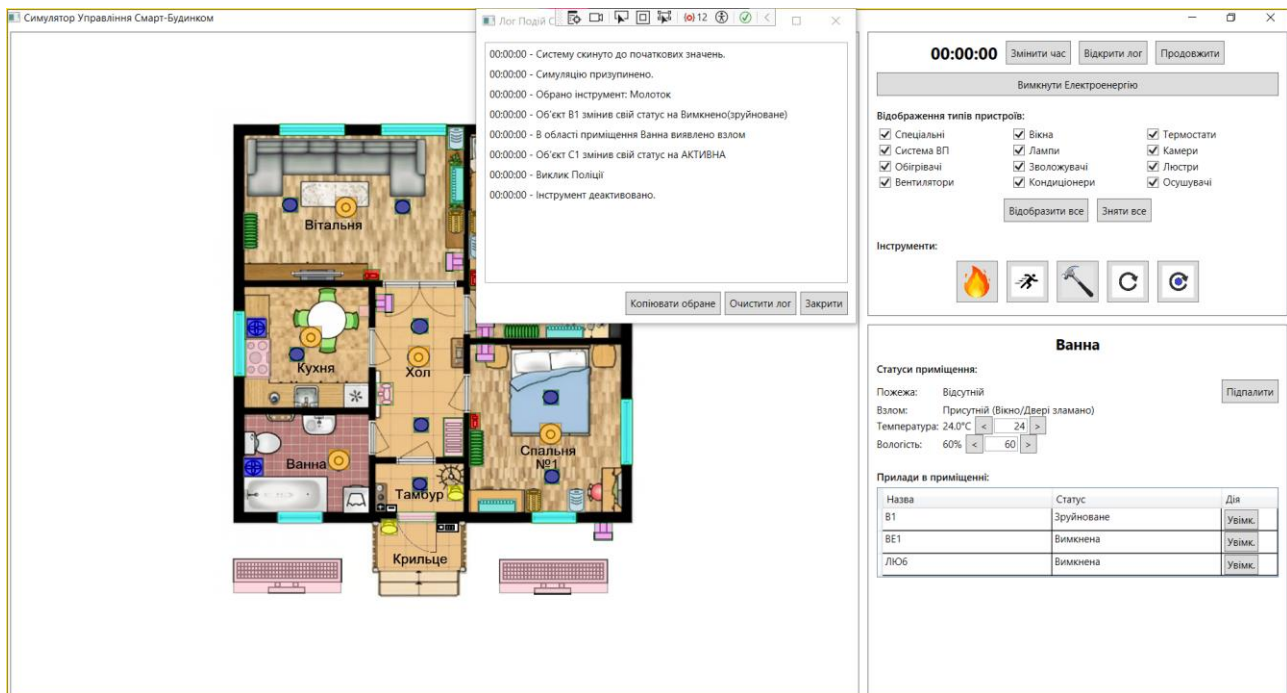


Рисунок 3.10. Відображення поточного стану об'єкту «В1» та області приміщення «Ванна» з вікном «Лог Подій Системи»

«Центральний контролер» бачить цю подію та автоматично надсилає команду активації на пристрій SirenDevice, який переходить у стан «Активна» та вмикає звуковий сигнал тривоги (рис. 3.11).

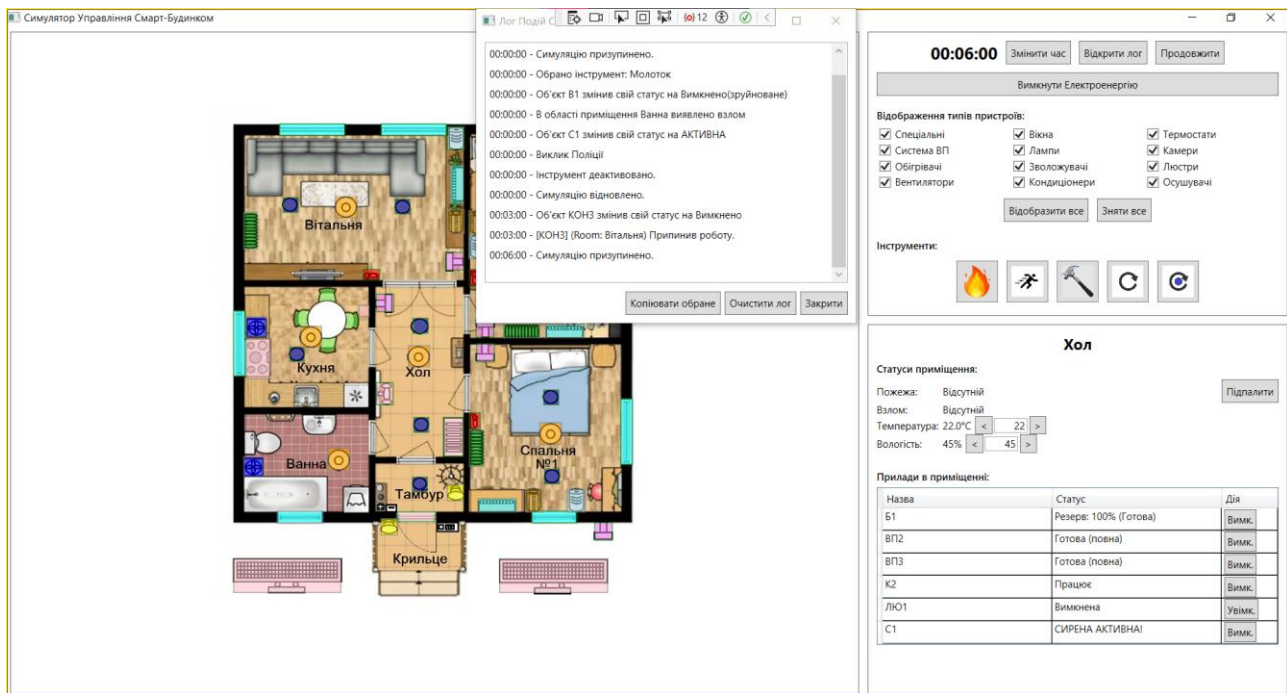


Рисунок 3.11. Відображення поточного стану об'єкту «Сирена» в області приміщення «Хол» з вікном «Лог Подій Системи»

Сценарій 6. Перевірка роботи системи автоматичного клімат-контролю. Початковий стан – температура у «Спальні 2» 22°C, вологість – 50%, термостат налаштований на 26°C та 66% вологості. «Обігрівач» HeaterDevice та «Зволожувач» HumidifierDevice автоматично вмикається (стан «Працює»), оскільки поточна температура та вологість нижчі за цільову (рис. 3.12).

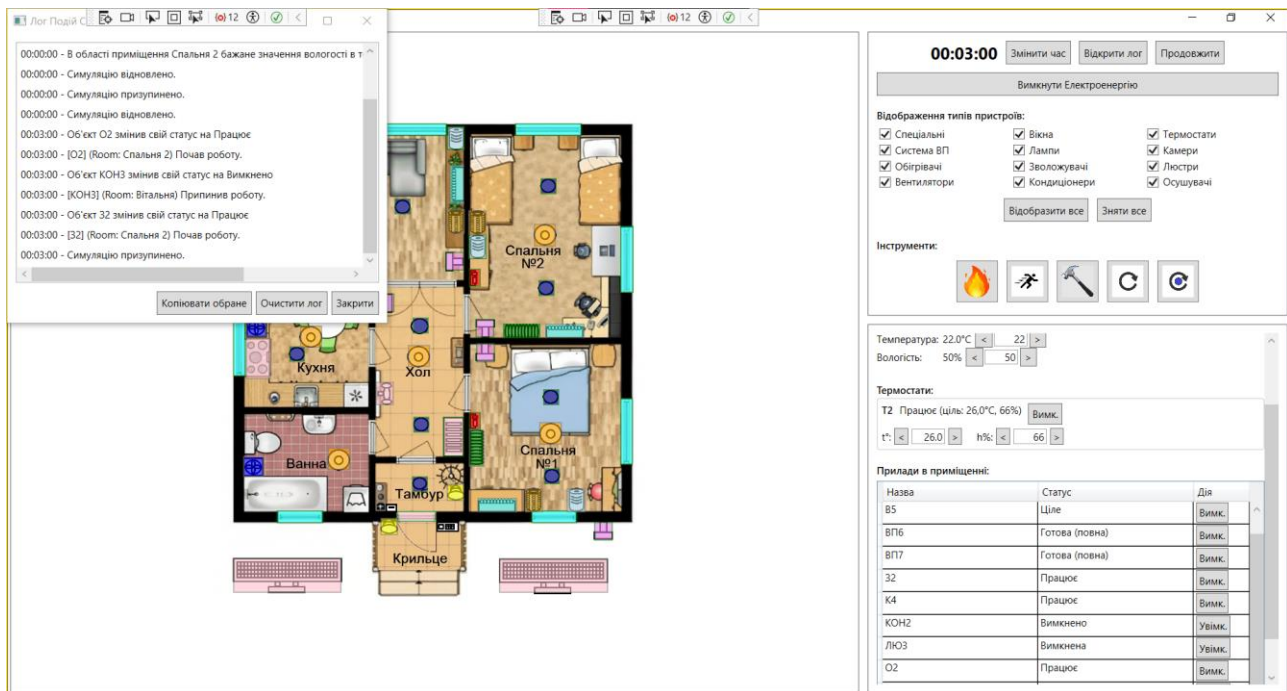


Рисунок 3.12. Відображення поточних станів температури та вологості в області приміщення «Спальня 2» з вікном «Лог Подій Системи»

З часом температура та вологість в області приміщення зростає. Коли температура досягає 26°C а вологість 66%, «Обігрівач» та «Зволожувач» автоматично переходить у режим очікування («Не працює»), що підтверджує коректну роботу алгоритму зворотного зв'язку (рис. 3.13).

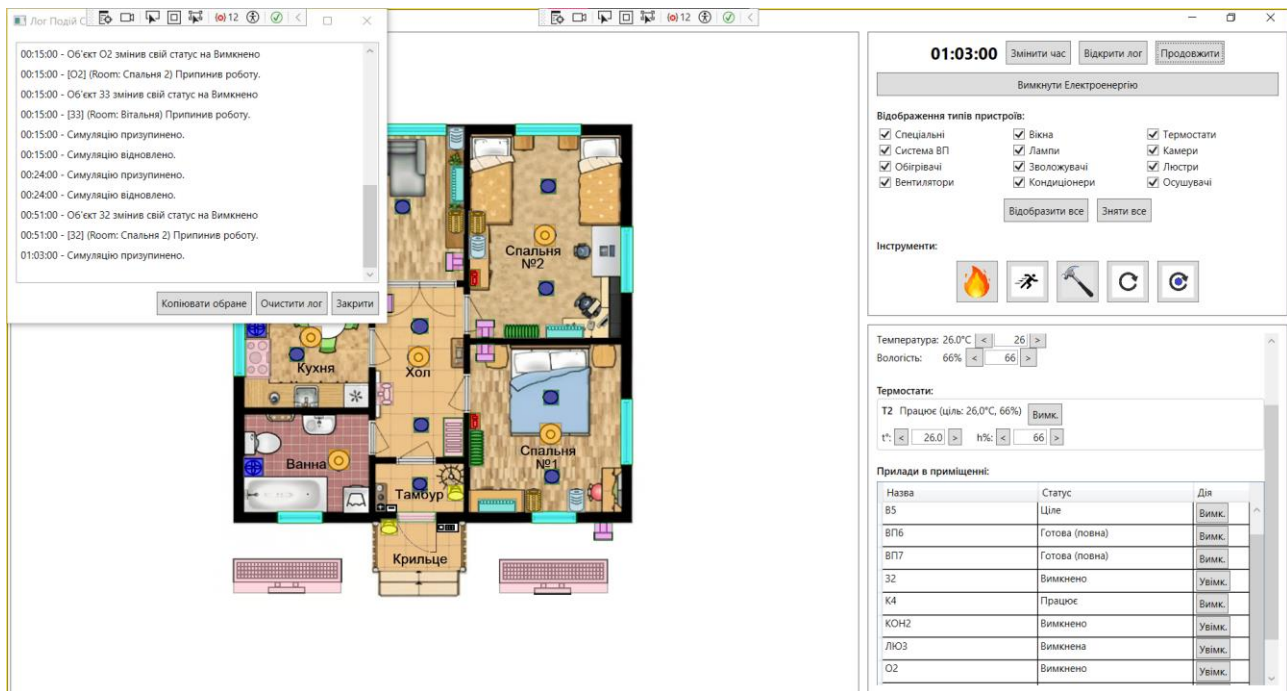


Рисунок 3.13. Відображення поточних станів температури, вологості та приладів в області приміщення «Спальня 2» з вікном «Лог Подій Системи»

3.5 Висновки до третього розділу

У цьому розділі здійснено програмну реалізацію симулятора системи управління смарт-будинком відповідно до спроектованої раніше архітектури. Обґрунтовано вибір мови програмування C# та платформи .NET як оптимальних засобів для створення об'єктно-орієнтованої моделі, а також технології WPF для побудови інтерактивного графічного інтерфейсу з використанням механізмів прив'язки даних Data Binding та векторної графіки.

В ході розробки було успішно реалізовано ключові компоненти системи – «Центральний контролер» з таймером симуляції, модель даних та ієрархію класів пристроїв. Практично застосовано принципи ООП, зокрема наслідування через базовий клас SmartDeviceBase, інкапсуляцію для захисту стану об'єктів та поліморфізм для уніфікованого оновлення станів різномірних пристроїв. Це забезпечило гнучкість коду та можливість легкого розширення функціоналу симулятора.

Проведене функціональне тестування за шістьма типовими сценаріями підтвердило працездатність розроблених алгоритмів. Зокрема, перевірено

коректність роботи головного циклу симуляції, логіки енергозалежності (перемикання на резервне живлення), реакції систем безпеки на рух та злом, а також автоматичне регулювання клімату та спрацювання системи пожежогасіння. Результати тестування показують, що програмний продукт стабільно виконує поставлені завдання, а візуалізація станів пристроїв на 2D-плані відповідає реальному часу симуляції.

Висновки

У результаті виконання курсової роботи було спроектовано та програмно реалізовано симулятор системи управління смарт-будинком. Пройдено повний цикл розробки програмного забезпечення: від аналізу предметної області та існуючих аналогів до побудови архітектури, написання коду та тестування готового продукту. Головним результатом роботи є функціонуючий настільний застосунок, який успішно імітує роботу комплексної системи автоматизації житла, базуючись на принципах об'єктно-орієнтованого програмування.

Розроблена система володіє широкими функціональними можливостями, що покривають ключові аспекти «розумного дому». Симулятор забезпечує динамічне моделювання часу з можливістю прискорення, що дозволяє ефективно тестувати тривалі процеси. Реалізовано інтерактивний 2D-план будинку з візуалізацією станів пристроїв у реальному часі. Система підтримує складну бізнес-логіку автоматизації для різних підсистем – клімат-контролю (автоматичне регулювання температури та вологості), безпеки (реакція на рух та несанкціоноване проникнення), пожежної безпеки (автоматичне гасіння) та енергозабезпечення (робота від мережі, акумуляторів та сонячних панелей). Користувачу надано набір інструментів як «Пожежа», «Злом», «Рухоме тіло» та інші для створення аварійних сценаріїв та перевірки реакції системи.

Для реалізації програмного продукту було обрано сучасні засоби розробки. В якості мови програмування використано C# на платформі .NET, що забезпечило надійність та гнучкість коду. Графічний інтерфейс побудовано на базі фреймворку Windows Presentation Foundation (WPF), що дозволило використати векторну графіку для плану будинку та механізм прив'язки даних (Binding) для чіткого розділення логіки та візуалізації. Середовищем розробки виступило Microsoft Visual Studio.

Практичне значення отриманої системи полягає у можливості моделювання та налагодження сценаріїв роботи розумного будинку без необхідності використання дороговартісного фізичного обладнання. Це дозволяє безпечно перевіряти логіку взаємодії пристроїв та виявляти потенційні проблеми

на етапі проектування. Перспективою подальшого розвитку та вдосконалення системи є інтеграція до її складу редактора планів будинків, що дозволить користувачам самостійно створювати та змінювати топологію приміщень для симуляції.

Список використаних джерел

1. Which Smart Home System is Best? A Look at Commercial and Open-Source Platforms. [Вебсайт]. URL: <https://www.binarytechlabs.com/which-smart-home-system-is-best/> (дата звернення: 05.04.2025).
2. Balancing walled garden and open platform approaches for the Internet of Things. [Електронний документ]. URL: <https://www.diva-portal.org/smash/get/diva2:1106581/FULLTEXT01.pdf> (дата звернення: 12.04.2025).
3. Amazon Alexa. [Вебсайт]. URL: <https://www.amazon.com/alexa> (дата звернення: 20.04.2025).
4. Google Assistant. [Вебсайт]. URL: <https://assistant.google.com/> (дата звернення: 28.04.2025).
5. Apple Home. [Вебсайт]. URL: <https://www.apple.com/ios/home/> (дата звернення: 07.05.2025).
6. Home Assistant. [Вебсайт]. URL: <https://www.home-assistant.io/> (дата звернення: 15.05.2025).
7. openHAB. [Вебсайт]. URL: <https://www.openhab.org/> (дата звернення: 22.05.2025).
8. Imperative vs Declarative UI Design – Is Declarative Programming the future?. [Вебсайт]. URL: <https://www.rootstrap.com/blog/imperative-v-declarative-ui-design-is-declarative-programming-the-future> (дата звернення: 30.05.2025).
9. Windows Forms documentation. [Вебсайт]. URL: <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/> (дата звернення: 06.06.2025).
10. The Swing Architecture. [Вебсайт]. URL: <https://docs.oracle.com/javase/tutorial/uiswing/concurrency/index.html> (дата звернення: 14.06.2025).
11. GDI+ Reference. [Вебсайт]. URL: <https://learn.microsoft.com/en-us/windows/win32/gdiplus/-gdiplus-gdi-start> (дата звернення: 21.06.2025).

12. Imperative vs. Declarative UI Development. [Вебсайт]. URL: <https://medium.com/icomunity/imperative-vs-declarative-ui-development-a-comparison-of-uikit-and-swiftui-4bc96b5d8684> (дата звернення: 29.06.2025).
13. Windows Presentation Foundation for .NET documentation. [Вебсайт]. URL: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/> (дата звернення: 05.07.2025).
14. JavaFX Documentation. [Вебсайт]. URL: <https://openjfx.io/openjfx-docs/> (дата звернення: 11.07.2025).
15. QML Applications. [Вебсайт]. URL: <https://doc.qt.io/qt-6/qmlapplications.html> (дата звернення: 19.07.2025).
16. Canvas Class (System.Windows.Controls). [Вебсайт]. URL: <https://learn.microsoft.com/en-us/dotnet/api/system.windows.controls.canvas> (дата звернення: 25.07.2025).
17. XAML overview (WPF .NET). [Вебсайт]. URL: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/xaml/> (дата звернення: 02.08.2025).
18. Data binding overview (WPF .NET). [Вебсайт]. URL: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/data/> (дата звернення: 09.08.2025).
19. Discrete-event simulation. [Вебсайт]. URL: https://en.wikipedia.org/wiki/Discrete-event_simulation (дата звернення: 16.08.2025).
20. Game Loop Pattern. [Вебсайт]. URL: <https://gameprogrammingpatterns.com/game-loop.html> (дата звернення: 24.08.2025).
21. Event-condition-action. [Вебсайт]. URL: https://en.wikipedia.org/wiki/Event_condition_action (дата звернення: 30.08.2025).
22. Finite-state machine. [Вебсайт]. URL: https://en.wikipedia.org/wiki/Finite-state_machine (дата звернення: 05.09.2025).

23. Avoiding Software Bottlenecks: The 'God Object' Anti-Pattern. [Вебсайт]. URL: <https://hackernoon.com/avoiding-software-bottlenecks-understanding-the-god-object-anti-pattern> (дата звернення: 12.09.2025).
24. OOP Concepts for Beginners: What Is Polymorphism. [Вебсайт]. URL: <https://stackify.com/oop-concept-polymorphism/> (дата звернення: 18.09.2025).
25. Imperative vs Declarative in Android – The Real Difference. [Вебсайт]. URL: <https://itnext.io/imperative-vs-declarative-in-android-the-real-difference-bd9bdce1c358> (дата звернення: 25.09.2025).
26. OOP. [Вебсайт]. URL: <https://metanit.com/sharp/tutorial/3.7.php> (дата звернення: 28.09.2025).
27. Built in reference types (справочник по C#). [Вебсайт]. URL: <https://learn.microsoft.com/ru-ru/dotnet/csharp/language-reference/builtin-types/collections> (дата звернення: 30.09.2025).
28. Inheritance in C# and .NET. [Вебсайт]. URL: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/inheritance> (дата звернення: 01.10.2025).
29. What is .NET?. [Вебсайт]. URL: <https://dotnet.microsoft.com/en-us/learn/dotnet/what-is-dotnet> (дата звернення: 07.10.2025).
30. Language Integrated Query (LINQ) (C#). [Вебсайт]. URL: <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/linq/> (дата звернення: 15.10.2025).
31. INotifyPropertyChanged Interface. [Вебсайт]. URL: <https://learn.microsoft.com/en-us/dotnet/api/system.componentmodel.inotifypropertychanged> (дата звернення: 21.10.2025).
32. Visual Studio documentation. [Вебсайт]. URL: <https://learn.microsoft.com/en-us/visualstudio/windows/> (дата звернення: 27.10.2025).

33. Design XAML in Visual Studio and Blend for Visual Studio. [Вебсайт]. URL: <https://learn.microsoft.com/en-us/visualstudio/xaml-tools/designing-xaml-in-visual-studio?view=visualstudio> (дата звернення: 29.10.2025).
34. DispatcherTimer Class. [Вебсайт]. URL: <https://learn.microsoft.com/en-us/dotnet/api/system.windows.threading.dispatchertimer?view=windowsdesktop-10.0> (дата звернення: 01.11.2025).

Додатки

Додаток А. Словник ключових термінів проєкту

1. Час симуляції – внутрішній віртуальний час системи, що спливає з прискоренням відносно реального часу. Масштаб часу: 1 реальна секунда дорівнює 6 хвилинам симуляційного часу. Використовується для моделювання тривалих процесів (нагрівання, охолодження, зміна доби) у стислі терміни.

2. Області приміщення – логічно виділені зони на 2D-плані будинку, що відповідають реальним кімнатам (Вітальня, Кухня, Спальня тощо). Кожна область має власні параметри середовища (температура, вологість, наявність пожежі/руху) та містить прив'язані до неї смарт-пристрої.

3. Області приладів та об'єктів – інтерактивні графічні елементи, розташовані поверх областей приміщень, що представляють конкретні смарт-пристрої (лампи, камери, вікна). Вони візуалізують стан пристрою через колір заливки та рамки і дозволяють користувачу взаємодіяти з ними (клікати для зміни стану або перегляду інформації).

4. Бажана температура та вологість – цільові значення кліматичних параметрів, які користувач задає вручну на пристрої типу "Термостат". Ці значення слугують орієнтиром для автоматичної роботи кліматичної техніки (обігрівачів, кондиціонерів, зволожувачів, осушувачів та вентилятора), яка намагається досягти цих показників у приміщенні.

5. Поточна температура та вологість – динамічні параметри стану конкретної області приміщення, що змінюються в часі під впливом працюючих приладів (наприклад, обігрівач підвищує температуру) або зовнішніх факторів. Саме ці значення відображають реальний стан мікроклімату в кімнаті на даний момент часу симуляції.

6. Стан пристрою – набір станів, що описують роботу пристрою (Off, Working, NotWorking, Destroyed, EmptyWater тощо). Визначає логіку поведінки та візуальне відображення.

7. Інструменти впливу – набір функціональних режимів курсору (наприклад "Пожежа", "Рухоме тіло", "Молоток"), що дозволяють користувачу

ініціювати зовнішні події та аварійні ситуації для перевірки реакції системи автоматизації.

8. Глобальний стан – параметри, що впливають на всю симуляцію, наприклад CurrentSimTime (поточний час), IsElectricityOn (наявність живлення) тощо.

9. «Легковаговий симулятор» – це програма, яка імітує певний процес або систему, використовуючи при цьому мінімум комп'ютерних ресурсів (процесорного часу, пам'яті, дискового простору) і часто має спрощений функціонал у порівнянні з повнофункціональними або «важкими» аналогами.

Додаток Б. Список усіх областей приладів та об'єктів

1. «Вікно» (WindowDevice) – інфраструктурний об'єкт, що відображає стан віконного отвору. Має два основних стани, наприклад, «Ціле» (працює) та «Зруйноване» (вимкнено). Зміна стану на «зруйноване» відбувається під дією інструменту «Молоток» (імітація злому), що, у свою чергу, є тригером для активації системи сигналізації.

2. «Лампа» (MotionSensorLamp) – освітлювальний прилад, оснащений сенсором присутності. Автоматично переходить у активний стан при використанні інструменту «Рухоме тіло» у відповідній області приміщення та залишається увімкненим протягом заданого часу (таймер зворотного відліку). Деякі моделі мають додаткове обмеження на роботу лише у нічний час (наприклад, з 16:00 до 8:00).

3. «Камера» (CameraDevice) – пристрій системи безпеки, що реагує на рух. При використанні інструменту «Рухоме тіло» переходить у режим запису на фіксований час, імітуючи відеофіксацію події в області приміщення в якій розташований. Має пріоритетне живлення і продовжує працювати від резервної батареї навіть при відключенні основної електромережі.

4. «Водяне пожежогасіння» (FireSprinklerDevice) – автоматичний спринклер, призначений для гасіння вогню. Активується при появі статусу «Пожежа» в області приміщення в якій розташований, спираючись на внутрішню ймовірність спрацювання (симуляція надійності). Успішна активація гасить пожежу, але переводить пристрій у стан "Пустий" (вичерпання води), після чого він потребує фіксованого часу на автоматичну або ручну перезарядку.

5. «Термостат» (ThermostatDevice) – керуючий прилад кліматичної системи. Дозволяє користувачу встановлювати бажані параметри мікроклімату – цільову температуру (у діапазоні 16-28°C) та вологість (20-80%). Слугує орієнтиром для інших кліматичних пристроїв і області приміщення, які зчитують з нього налаштування.

6. «Обігрівач» (HeaterDevice) – кліматичний прилад для підвищення температури. Автоматично вмикається, якщо поточна температура в області

приміщення нижча за бажану, задану в термостаті, і поступово підвищує її до досягнення бажаного значення.

7. «Кондиціонер» (ConditionerDevice) – кліматичний прилад для охолодження. Працює за логікою, зворотною до обігрівача – активується, коли температура в області приміщення перевищує задану бажану в термостаті, і знижує її.

8. «Зволожувач» (HumidifierDevice) – прилад для контролю вологості. Вмикається автоматично, якщо поточний рівень вологості в приміщенні нижчий за бажаний, встановлений на термостаті.

9. «Осушувач» (DehumidifierDevice) – прилад для зменшення вологості. Активується, коли вологість в області приміщення перевищує задану бажану в термостаті, запобігаючи надмірній вологості.

10. «Люстра» (ChandelierDevice) – стандартне джерело світла. Керується виключно вручну користувачем (увімкнення/вимкнення через інтерфейс). Робота приладу повністю залежить від наявності електроенергії в мережі.

11. «Вентилятор» (FanDevice) – пристрій для примусової вентиляції. Може працювати в автоматичному режимі (наприклад, у ванній кімнаті при перевищенні порогу вологості 60%) або керуватися вручну та іншими приладами (на кухні вмикається разом із плитою). Знижує значення вологості в області приміщення де розташований.

12. «Сонячна» панель (SolarPanelDevice) – джерело відновлюваної енергії. Генерує електроенергію залежно від часу доби (активна у світлий період, наприклад з 8:00 до 16:00). Дозволяє жити систему або заряджати акумулятор у денний час.

13. «Сирена» (SirenDevice) – звуковий оповісник системи безпеки. Не має власних сенсорів, а активується за командою від «Центрального Контролера» при настанні критичних подій – використанні інструмента «Молоток» (для статусу «Злом») на вікна, двері або тривалій пожежі.

14. «Двері» (DoorDevice) – об'єкт, аналогічний вікнам. Може перебувати у стані «Цілі» або «Зруйновані». Злом дверей є тригером тривоги.

15. «Батарея» (BatteryDevice) – акумуляторна система живлення. Накопичує заряд при наявності електромережі або активних сонячних панелей і віддає енергію критично важливим пристроям (камери, сирена, пожежна система та ін.) у разі знеструмлення будинку.

16. «Перемикач» (ManualSwitchDevice) – механічний елемент керування. Дозволяє вручну активувати специфічні пристрої (наприклад, кухонний спринклер), які не мають автоматичного режиму спрацювання.

17. «Плита» (StoveDevice) – побутовий прилад. Має ручне керування і логічно пов'язаний з кухонним вентилятором (автоматично вмикає витяжку при роботі).

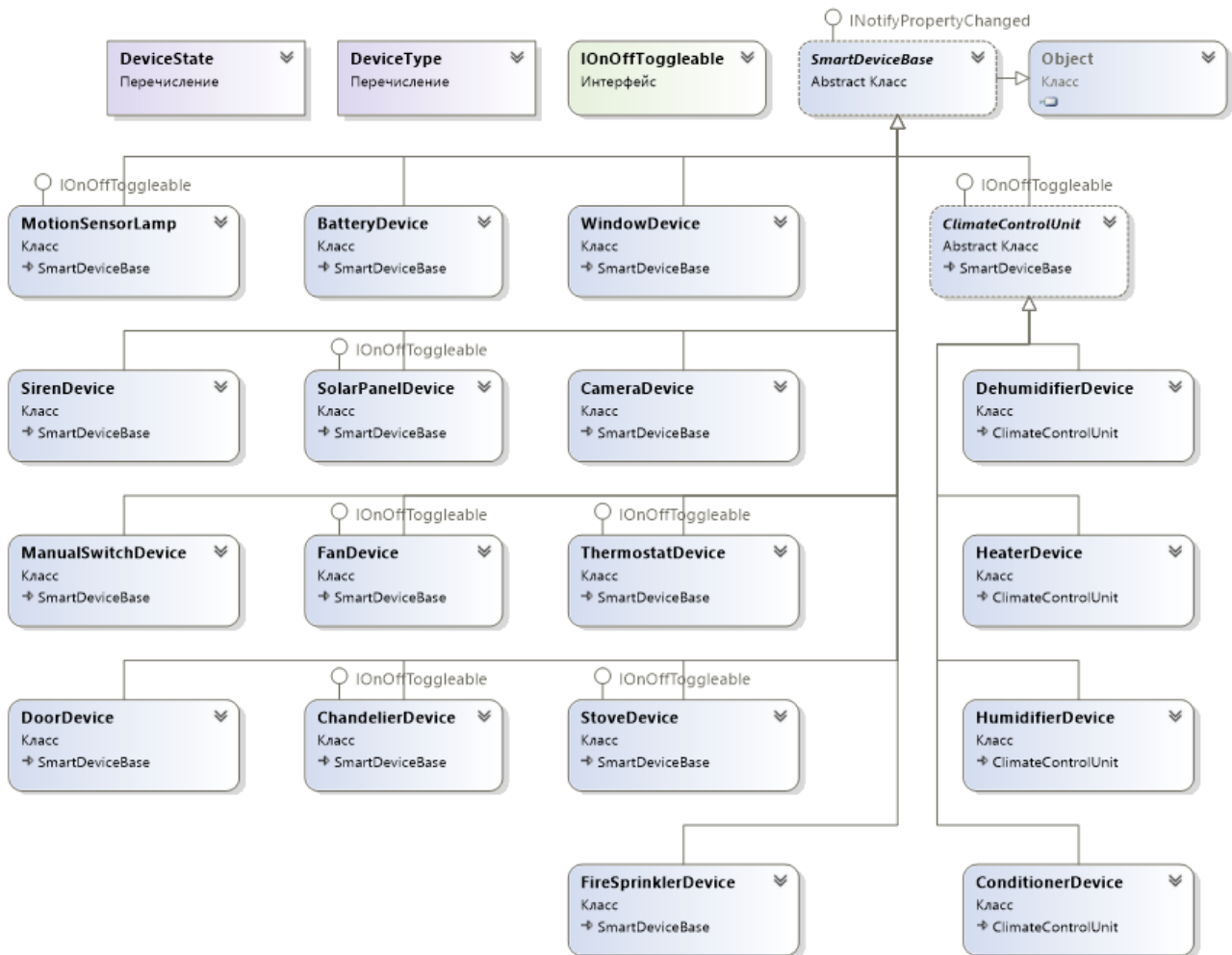
Додаток В. Повний код проекту

Повний код проекту наведений в репозиторії гітхабу за даним посиланням:

<https://github.com/IvanTukalo/CourseWork/tree/New-pc>

Додаток Г. Діаграми класів

Діаграма класів, наслідуваних від SmartDeviceBase, переліки та інтерфейси:



Діаграма усіх інших класів:

